
Hierarchical Bayesian Patch Time Series Transformer

Zachary Olea

zolea1@jh.edu

Master's Thesis

Applied & Computational Mathematics

JHU Whiting School of Engineering EP

November 2, 2025

Abstract

Forecasting hierarchical time series requires capturing cross-series interactions without overfitting while keeping forecasts coherent across aggregation levels. This paper will propose Hierarchical Bayesian PatchTST, a patch-based Transformer with a factorized design: products, stores, and areas are encoded with shared weights, then composed into latent leaf-level forecasts through a low-rank tensor interaction head. A Bayesian front end imposes structured inductive biases, (i) temporally decaying Gaussian priors on patch-embedding weights to privilege recent information, and (ii) Minnesota-style shrinkage on within-group mixers to favor own-series signals while permitting evidence-based borrowing. Static covariates enter via feature-wise linear modulation (FiLM) and a lightweight horizon-wise bias head, injecting entity semantics and smooth shape priors without inflating the encoder.

Hierarchical coherence is enforced during training through a weighted hierarchical likelihood: latent leaf forecasts are aggregated by known summation matrices and compared to observed parent series (soft reconciliation), with group-specific weights emphasizing stable aggregates. Estimation uses Stochastic Variational Inference, scaling to long horizons and many series while yielding posterior predictive uncertainty across hierarchy levels. The factorized, low-rank head avoids quadratic parameter growth from dense channel mixing, improving statistical efficiency.

We evaluate on a controlled retail hierarchy with a one-year horizon, reporting MAPE at bottom and aggregate levels and benchmarking against classical VAR/VARX and deep PatchTST baselines. Results and ablations show that multi-stage Bayesian pooling, low-rank composition, and soft reconciliation produce accurate, interpretable, and hierarchy-consistent forecasts while preserving the efficiency benefits of patch-based encoders.

Contents

Contents	i
1 Introduction	1
1.1 Motivation and Problem Statement	1
1.2 The Gap in Existing Methodologies	2
1.3 Proposed Model and Core Innovation	3
1.4 Contributions and Thesis Outline	4
2 Data Structure and Notation	6
2.1 Time Series	6
2.1.1 Univariate Time Series	6
2.1.2 Collections of Univariate Series	6
2.1.3 Multivariate Time Series	7
2.2 Panel Data	7
2.2.1 Balanced Panel	7
2.2.2 Unbalanced Panel	7
2.3 Structured Data Formats	8
2.3.1 Long Format	8
2.3.2 Wide Format	8
2.3.3 Tensor Format	8
2.3.4 Mixed Format	8
2.4 Hierarchical Data	9
2.4.1 General Multilevel Structure	9
2.4.2 Hierarchical Time Series	9
2.4.3 Hierarchical Panel Data	9
2.4.4 Cross-Hierarchical (Multi-Dimensional) Structures	9
2.5 Example	10

3	Statistical and Econometric Models	11
3.1	Overview and Motivation	11
3.2	Univariate and Multivariate Regression Models	11
3.2.1	Autoregressive Models (AR)	11
3.2.2	Vector Autoregressions (VAR) and VARX	12
3.3	Unit Heterogeneity and Pooling (Panels)	12
3.3.1	Fixed Effects (Within-Entity Intercepts)	12
3.3.2	Random Effects and Partial Pooling	13
3.3.3	Pooling Taxonomy and a Block-Diagonal Example	13
3.3.4	Mixed-Effects VAR (Unified Template)	13
3.4	Bayesian VAR in High Dimensions	14
3.4.1	Why Shrinkage?	14
3.4.2	Structured Shrinkage: The Minnesota Prior	14
3.4.3	Alternative Shrinkage Priors (Briefly)	15
3.5	Innovation Covariance Σ (Forecasting View)	15
3.6	Summary and Link to the Proposed Model	15
4	Deep Neural Networks for Time Series	16
4.1	Recurrent Neural Networks (RNNs)	16
4.2	Long Short-Term Memory (LSTM) Networks	17
4.3	The Transformer Framework	18
4.4	Patch Time Series Transformer (PatchTST)	19
4.4.1	Patch Extraction	19
4.4.2	Patch Embedding and Positional Encodings	20
4.4.3	Transformer Encoder	20
4.4.4	Feature Representation	21
4.4.5	Prediction Head and Training Objective	21
4.4.6	Why Patches Help	21
4.4.7	Model Rationale and Limitations	22
4.4.8	Architectural Summary and Schematic	22
5	Proposed Model: Hierarchical Bayesian PatchTST	23
5.1	Core Architecture: A Factorized Approach to Forecasting	24
5.1.1	Unified Hierarchical Structure	24
5.1.2	Bayesian Front-End: Structured Feature Extraction	24
5.1.3	The Core Encoder	25
5.1.4	Static Covariate Injection	26

5.1.5	Low-Rank Forecast Composition	26
5.2	Hierarchical Coherence and Inference	27
5.2.1	Soft Reconciliation via a Weighted Hierarchical Likelihood	27
5.2.2	Probabilistic Inference via SVI	28
5.3	Model Complexity and Scalability	28
5.4	Algorithmic Specification	29
5.5	Model Implementation with PyTorch and Pyro	30
5.5.1	The Bayesian Front-End	30
5.5.2	Covariate Injection and Feature Modulation	32
5.5.3	Low-Rank Forecast Composition	32
5.5.4	Weighted Likelihood and Latent Variable Supervision	33
5.5.5	Inference via SVI	33
5.5.6	Summary of Design Choices	34
6	Simulation Study	35
6.1	The Data Generating Process	35
6.1.1	Temporal Dynamics and Stochastic Components	35
6.1.2	Structural Heterogeneity and Product-Level Dynamics	36
6.1.3	Observational Noise Model	38
6.2	Evaluation Protocol	38
6.2.1	Metrics and Aggregation Order	38
6.2.2	Implementation Notes	40
6.3	Benchmark Model 1: VAR	40
6.4	Benchmark Model 2: Vanilla PatchTST	41
6.5	Results, Comparison, and Discussion	42
6.5.1	Metric Table Comparison	43
6.5.2	Other Tables and Plots	46
7	Discussion and Conclusion	48
7.1	Strengths and Limitations	48
7.2	Summary of Contributions	49
7.3	Implications for Practice and Theory	50
7.4	Recommendations for Future Directions	50
	Bibliography	51
A	Data Generating Process	54

Chapter 1

Introduction

1.1 Motivation and Problem Statement

Accurate multivariate time-series forecasting underpins decisions in economics, finance, and supply chains. Real-world datasets are seldom collections of independent series; they are panels with hierarchical structure. Such structure imposes (i) static constraints (e.g., additive aggregation of product sales to category totals) and (ii) dynamic dependencies (e.g., substitution and cannibalization across related products). The modeling challenge is to capture cross-series interactions while respecting these constraints.

Classical econometric models such as vector autoregressions (VARs) provide a principled way to encode linear interdependencies, but scale poorly: with C series and lag order p , parameter counts grow as $O(C^2p)$, inviting overfitting unless strong structure (e.g., Minnesota priors) is imposed. Modern deep architectures, especially Transformers, excel at learning long-range, nonlinear patterns, yet their leading time-series instantiation (PatchTST) attains efficiency by a channel-independent design that processes each series in isolation, precluding explicit cross-series learning. The opposite extreme, full channel mixing, is maximally expressive but reintroduces $O(C^2)$ complexity in parameters and compute, which is brittle in high-dimensional panels.

This leaves a gap, neither the scalable but cross-blind channel-independent approach nor the expressive but unwieldy fully connected approach offers a satisfactory solution when cross-series effects matter. The motivation of this thesis is to bridge that gap by unifying the representational power of Transformers with the principled regularization of Bayesian econometrics, enabling the model to learn data-supported cross-channel interactions while shrinking weak links toward a parsimonious structure, achieving the benefits of mixing without sacrificing statistical efficiency or scalability.

1.2 The Gap in Existing Methodologies

The modeling of structured multivariate time series sits at the intersection of two mature traditions, classical econometrics and modern deep learning, each providing compelling but incomplete solutions. Econometric formulations (e.g., state space and vector autoregressions) offer interpretable dynamics, well-understood identification strategies, and principled uncertainty quantification. Yet they rely on linearity (or limited nonlinear extensions) and scale unfavorably as the cross-sectional dimension grows. Deep neural architectures, especially Transformers, excel at capturing long-range, nonlinear dependencies in high-dimensional sequences with minimal feature engineering, but they often sacrifice clear inductive structure and can be difficult to regularize in panel settings with limited effective sample sizes.

Within the econometric paradigm, vector autoregressions (VARs) remain a workhorse for modeling cross-series dynamics. With C series and lag order p , unrestricted $\text{VAR}(p)$ parameter counts scale as $O(C^2p)$, producing brittle estimates and wide forecast dispersion without strong structure or shrinkage (1). Bayesian regularization has long been the remedy: the Minnesota prior and its descendants shrink coefficients toward random-walk or own-lag baselines, stabilizing estimation and enabling “large” VARs in practice (2; 3). Related hierarchical Bayes formulations deliver partial pooling across related series, improving out-of-sample performance when cross-sectional signal is heterogeneous (4; 5). These advances, however, remain largely linear in their treatment of dependence and require careful prior engineering to avoid either under- or over-shrinking salient interactions.

Deep learning approaches reframe multivariate forecasting as representation learning. Temporal Fusion Transformers (TFT) combine recurrent/attention blocks with gating and static encoders, targeting multi-horizon accuracy and some degree of interpretability (6). More recent architectures replace tokenization at the time-step level with patch tokens and show strong long-horizon performance (7). A key design fork arises for multivariate panels: channel-independent (CI) models share weights across series and process each channel as a univariate input, prioritizing statistical efficiency and generalization; by construction they cannot learn explicit cross-channel dependencies. At the other extreme, channel-mixing designs (e.g., dense cross-channel layers or cross-dimension attention) are maximally expressive but reintroduce $O(C^2)$ parameter and compute growth, increasing overfitting risk and training cost on large panels (8). Empirically, CI baselines such as PatchTST are strong and robust, while mixing models show gains primarily when cross-variate signal is strong relative to noise, precisely the regime where regularization choices become consequential.

A second and orthogonal axis is hierarchical coherence. In retail, demand planning, and macroeconomic applications, series live in trees or grouped structures where linear aggrega-

tion constraints must hold. Forecast-reconciliation methods address this by generating base forecasts and then projecting them onto the coherent subspace via generalized least squares, yielding the now-standard MinT family and its variants (9; 10). Reconciliation demonstrably improves accuracy in large hierarchies and delivers coherent decisions across levels, but it is typically an ex post step that does not, by itself, endow the forecasting model with inductive biases about cross-series interactions or selective sharing of statistical strength (9; 10). Neural architectures explicitly designed for hierarchies (e.g., NHITS) inject multi-scale structure and often pair naturally with reconciliation, while recent Transformer variants attach hierarchical modules or distillation to capture cross-level information (11; 12; 13; 14; 15; 16).

These strands reveal a persistent methodological gap. Classical Bayesian models provide principled shrinkage for linear cross-series dynamics at $O(C^2)$ complexity, while modern Transformers provide flexible sequence encoders but either (i) avoid cross-series structure (CI) and risk structural blindness, or (ii) embrace full mixing and inherit the same quadratic explosion that motivated Bayesian shrinkage in VARs. Bridging this gap requires frameworks that (a) represent nonlinear, long-range temporal patterns; (b) selectively learn cross-channel effects only when supported by data; (c) regularize those effects with explicit, hierarchy-aware priors; and (d) encourage coherence across aggregation levels during training rather than only via post-hoc projection. The approach developed in this thesis proceeds precisely along these lines: combining patch-based Transformer encoders with Bayesian partial pooling and soft hierarchical reconciliation so that significant interactions are retained, weak ones are shrunk, and coherent structure is respected without incurring the statistical and computational liabilities of fully dense mixing.

1.3 Proposed Model and Core Innovation

This thesis introduces the Hierarchical Bayesian PatchTST, a forecasting architecture that reconciles the long-standing tension between channel-independent encoders, which scale and generalize well but cannot learn explicit cross-series structure, and fully mixed multivariate models, which are expressive but grow statistically and computationally burdensome as the number of series increases.

The model is built around a factorized design: time series are organized into meaningful factor groups (e.g., items, locations, channels), encoded with a shared patch-based Transformer, and composed into leaf-level forecasts through a low-rank interaction head. A Bayesian front-end stabilizes feature extraction by placing structured priors directly where they matter most: (i) temporally decaying Gaussian priors on patch-embedding weights to privilege recent information, and (ii) Minnesota-style shrinkage on within-group mixer

matrices to favor own-series signals while permitting data-supported cross-series borrowing.

Static metadata enter through feature-wise modulation (FiLM) and a lightweight horizon-wise bias head, injecting entity semantics and smooth shape priors without inflating the temporal encoder. Forecasts are then produced by a low-rank composition over the forecast horizon, replacing dense channel mixing with a parameter-efficient tensor interaction that captures salient factor interactions.

Crucially, hierarchical structure is not an afterthought: the learning objective implements soft reconciliation via a weighted hierarchical likelihood. Leaf forecasts are treated as latent variables that must aggregate (via known summation matrices) to match observed parents, with group-specific weights emphasizing the most reliable aggregates. Estimation proceeds with stochastic variational inference (SVI), yielding scalable training and posterior predictive uncertainty across levels. In short, the model preserves the robustness of channel-independent encoders, selectively learns cross-series structure when warranted, and trains in a way that is both hierarchy-aware and probabilistically coherent.

1.4 Contributions and Thesis Outline

Contributions. The thesis advances multivariate, hierarchy-aware forecasting along four axes:

- **Architecture:** A factorized PatchTST with a low-rank composition head that captures cross-factor interactions without the quadratic growth of fully dense mixing.
- **Bayesian regularization:** Structured priors at the front-end, temporal-decay patch embeddings and Minnesota-style mixers, provide principled partial pooling that defaults to own-series behavior while enabling evidence-based sharing.
- **Hierarchy-aware training:** A weighted hierarchical likelihood supervises latent leaves through their aggregates (soft reconciliation), improving multi-level coherence and decision consistency.
- **Scalable uncertainty & evidence:** SVI delivers posterior predictive distributions at all hierarchy levels, supporting risk-aware planning in high-dimensional panels, with empirical validation against econometric and Transformer baselines in a controlled simulation testbed.

Thesis outline. Chapter 2 formalizes data structures and notation for panel and hierarchical time series. Chapter 3 reviews statistical and econometric models for time series,

emphasizing pooling and shrinkage principles that motivate our design choices. Chapter 4 surveys deep neural network approaches to time series, providing the ML background and design patterns we build upon. Chapter 5 presents the proposed model’s architecture and core design decisions, including the selective mixing mechanism and hierarchy-aware training objective and details the variational inference procedure and posterior predictive computation. Chapter 6 describes the simulation design and evaluation protocol and reports empirical results and ablations. Chapter 7 concludes with limitations and directions for future research.

Chapter 2

Data Structure and Notation

Conventions. We index time by $t \in \mathcal{T} = \{t_1, \dots, t_T\}$ (not necessarily equally spaced); variables/channels in a multivariate series by $j \in [C]$; and entities (e.g., stores) in a panel by $i \in [N]$. Bold lowercase denotes vectors ($\mathbf{y}$), bold uppercase matrices ($\mathbf{Y}$), and calligraphic letters tensors/sets ($\mathcal{Y}, \mathcal{I}$).

2.1 Time Series

2.1.1 Univariate Time Series

A univariate time series is a sequence of scalars,

$$\{y_t\}_{t \in \mathcal{T}}, \quad y_t \in \mathbb{R}, \quad (2.1)$$

often represented as a column vector $\mathbf{y} = [y_{t_1}, \dots, y_{t_T}]^\top \in \mathbb{R}^T$.

2.1.2 Collections of Univariate Series

For N semantically related but statistically distinct series,

$$\mathcal{G} = \{\mathbf{y}^{(i)} = [y_{i,t_1}, \dots, y_{i,t_{T_i}}]^\top \mid i \in [N]\}, \quad (2.2)$$

with either equal lengths ($T_i = T$, “balanced”) or varying lengths (T_i differs, “unbalanced”).

2.1.3 Multivariate Time Series

A multivariate series records C variables jointly over time:

$$\mathbf{y}_t = \begin{bmatrix} y_{1,t} \\ \vdots \\ y_{C,t} \end{bmatrix} \in \mathbb{R}^C, \quad \mathbf{Y}_{\text{multi}} = \begin{bmatrix} y_{1,t_1} & \cdots & y_{C,t_1} \\ \vdots & \ddots & \vdots \\ y_{1,t_T} & \cdots & y_{C,t_T} \end{bmatrix} \in \mathbb{R}^{T \times C}. \quad (2.3)$$

Here rows index time and columns index variables (channels).

2.2 Panel Data

2.2.1 Balanced Panel

A balanced panel observes every entity–time pair:

$$\mathcal{I}_{\text{bal}} = [N] \times \mathcal{T} = \{(i, t) : i \in [N], t \in \mathcal{T}\}. \quad (2.4)$$

Long format:

$$\mathcal{D}_{\text{bal}} = \{(i, t, \mathbf{x}_{i,t}, y_{i,t})\}_{(i,t) \in \mathcal{I}_{\text{bal}}}, \quad (2.5)$$

and equivalent wide form $\mathbf{Y}_{\text{bal}} \in \mathbb{R}^{N \times T}$ with $(\mathbf{Y}_{\text{bal}})_{i,\tau} = y_{i,t_\tau}$ (rows = entities; columns = times).

2.2.2 Unbalanced Panel

When some pairs are unobserved,

$$\mathcal{I} \subseteq [N] \times \mathcal{T}, \quad \mathcal{D}_{\text{unbal}} = \{(i, t, \mathbf{x}_{i,t}, y_{i,t}) : (i, t) \in \mathcal{I}\}. \quad (2.6)$$

Define an observation mask

$$\mathbf{M}^{\text{obs}} \in \{0, 1\}^{N \times T}, \quad m_{i,\tau}^{\text{obs}} = 1 \text{ iff } (i, t_\tau) \in \mathcal{I}. \quad (2.7)$$

For different types of missingness, random failures (MCAR) and selection-driven omissions (MNAR) differ from structurally undefined entries (e.g., an entity that does not yet exist). The latter are not latent values to impute; they should be excluded via a structural mask $\mathbf{M}^{\text{struct}}$ and omitted from both training losses and evaluation denominators.

2.3 Structured Data Formats

2.3.1 Long Format

Row-wise records indexed by keys (e.g., entity, time):

$$\mathcal{D}_{\text{long}} = \{(i, t, \mathbf{x}_{i,t}, y_{i,t}) : (i, t) \in \mathcal{I}\}. \quad (2.8)$$

We use “long,” “tabular,” and “flattened” interchangeably.

2.3.2 Wide Format

Pivot to a rectangular array:

$$\mathbf{Y} = [y_{i,t\tau}]_{i=1,\dots,N; \tau=1,\dots,T}, \quad \mathbf{M}^{\text{obs}} = [m_{i,\tau}^{\text{obs}}], \quad (2.9)$$

with NA where $m_{i,\tau}^{\text{obs}} = 0$. Wide form is natural for balanced panels and remains convenient for masked linear-algebra operations.

2.3.3 Tensor Format

Higher-order arrays for deep learning backends:

$$\mathcal{Y} \in \mathbb{R}^{N \times T \times P}, \quad (2.10)$$

e.g., batch/entity $\times$ time $\times$ features. Masks should mirror shapes (and, if needed, separate observation vs. structural masks).

2.3.4 Mixed Format

Combine tensorized temporal data with tabular metadata:

- $\mathcal{Y} \in \mathbb{R}^{N \times T \times D}$ (temporal features),
- $\mathcal{M} = \{(i, \text{region}_i, \text{product}_i, \text{category}_i)\}$ (entity metadata).

This supports learned embeddings and hierarchy-aware grouping while keeping sequence compute contiguous.

2.4 Hierarchical Data

2.4.1 General Multilevel Structure

Let $y_{i_1, \dots, i_L, t}$ index L nested levels (e.g., item $\rightarrow$ subcategory $\rightarrow$ category) and time t . Long form uses tuples $(i_1, \dots, i_L, t)$; tensor form stacks along axes $\mathcal{Y} \in \mathbb{R}^{N_1 \times \dots \times N_L \times T}$. Distinguish prescribed taxonomies (explicit metadata) from latent behavioral structure (similar seasonality, demand shocks), which can guide partial pooling.

2.4.2 Hierarchical Time Series

Additive constraints hold within each tree:

$$y_{r,t} = \sum_{s \in \mathcal{S}_r} y_{s,t}, \quad y_{s,t} = \sum_{i \in \mathcal{I}_s} y_{i,t}. \quad (2.11)$$

Let b be the number of bottom nodes and n total nodes. With the summing matrix $\mathbf{S} \in \{0, 1\}^{n \times b}$,

$$\mathbf{y}_t = \mathbf{S} \mathbf{y}_t^{(\text{bottom})}, \quad \mathbf{Y} \in \mathbb{R}^{n \times T}, \quad \mathbf{M} \in \{0, 1\}^{n \times T}. \quad (2.12)$$

2.4.3 Hierarchical Panel Data

Panel indices with hierarchy over entities:

$$\{y_{i,g,t} : i \in [N_g], g \in [G], t \in \mathcal{T}_{i,g}\}, \quad \mathcal{Y}_{\text{hier-panel}} \in \mathbb{R}^{G \times N \times T} \text{ (balanced)}. \quad (2.13)$$

For unbalanced cases, work with sparse index sets and masks.

2.4.4 Cross-Hierarchical (Multi-Dimensional) Structures

Independent hierarchies (e.g., locations and products) form a grid:

$$\mathcal{Y} \in \mathbb{R}^{|\mathcal{L}| \times |\mathcal{P}| \times T}, \quad \mathbf{S}^{(\text{loc})} \in \{0, 1\}^{n_{\text{geo}} \times |\mathcal{L}|}, \quad \mathbf{S}^{(\text{prod})} \in \{0, 1\}^{n_{\text{prod}} \times |\mathcal{P}|}. \quad (2.14)$$

At time t , aggregate via

$$\mathbf{Y}_t^{(\text{agg})} = \mathbf{S}^{(\text{loc})} \mathcal{Y}_{:,t} (\mathbf{S}^{(\text{prod})})^\top, \quad (2.15)$$

or, equivalently, in vectorized form with a Kronecker product,

$$\text{vec}(\mathbf{Y}_t^{(\text{agg})}) = (\mathbf{S}^{(\text{prod})} \otimes \mathbf{S}^{(\text{loc})}) \text{vec}(\mathcal{Y}_{::t}). \quad (2.16)$$

This non-nested structure is typical in retail (geo $\times$ product) and requires coherence across both hierarchies.

2.5 Example

Consider a retailer with transactions (`timestamp`, `store`, `product`, `amount`). Aggregating amounts daily yields a univariate $\{y_t\}$. Splitting by store gives a collection $\{y_{i,t}\}$ (balanced or unbalanced). Pairing sales with costs gives a multivariate series per entity. Geography (store $\rightarrow$ city $\rightarrow$ state $\rightarrow$ country) and product taxonomy (item $\rightarrow$ subcategory $\rightarrow$ category) induce two independent hierarchies. The resulting tensor $\mathcal{Y} \in \mathbb{R}^{L \times P \times T}$ supports cross-hierarchical aggregation using $\mathbf{S}^{(\text{loc})}$ and $\mathbf{S}^{(\text{prod})}$, while masks separate genuinely missing entries from structurally undefined periods. This representation underpins the modeling choices developed in later chapters.

Chapter 3

Statistical and Econometric Models

3.1 Overview and Motivation

Classical statistical and econometric models provide a principled vocabulary for handling dependence, heterogeneity, and structural constraints in time series. While traditionally emphasized for explanation and inference, we focus here on their utility for forecasting. In particular, we highlight three ideas that will reappear in later chapters: (i) multivariate dependence modeling (from AR to VAR/VARX), (ii) pooling across related units in panels (fixed, random, and mixed effects), and (iii) shrinkage to tame high-dimensional parameter spaces (Bayesian VARs with structured priors). Each of these will inform the design of the deep model proposed in Chapter 5.

Throughout, we use the conventions set in Chapter 2: time is indexed by $t \in \mathcal{T} = \{t_1, \dots, t_T\}$, multivariate variables/channels are indexed by $j \in [C]$, and panel entities are indexed by $i \in [N]$. Vectors are bold lowercase (e.g., $\mathbf{y}_t$), matrices bold uppercase (e.g., $\mathbf{Y}$).

3.2 Univariate and Multivariate Regression Models

3.2.1 Autoregressive Models (AR)

A basic univariate forecasting model of order p is

$$y_t = c + \sum_{k=1}^p \phi_k y_{t-k} + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, \sigma^2). \quad (3.1)$$

Deterministic terms (e.g., trend, seasonal dummies, holiday indicators) and exogenous regressors can be appended straightforwardly. The $\text{AR}(p)$ serves as a strong baseline but treats

the series in isolation.

3.2.2 Vector Autoregressions (VAR) and VARX

To model C series jointly, write the C -vector $\mathbf{y}_t = [y_{1,t}, \dots, y_{C,t}]^\top$ and specify

$$\mathbf{y}_t = \mathbf{c} + \sum_{k=1}^p \Phi_k \mathbf{y}_{t-k} + \mathbf{B} \mathbf{x}_t + \boldsymbol{\varepsilon}_t, \quad \boldsymbol{\varepsilon}_t \sim \mathcal{N}(\mathbf{0}, \Sigma), \quad (3.2)$$

where $\mathbf{c} \in \mathbb{R}^C$ is an intercept, $\Phi_k \in \mathbb{R}^{C \times C}$ are lag coefficients, $\mathbf{x}_t$ collects exogenous regressors and/or deterministic terms, and $\mathbf{B}$ their loadings. Stacking the lags into $\mathbf{X}_t = [\mathbf{y}_{t-1}^\top, \dots, \mathbf{y}_{t-p}^\top]^\top \in \mathbb{R}^{Cp}$ and concatenating $\Phi = [\Phi_1 \dots \Phi_p] \in \mathbb{R}^{C \times Cp}$ gives the familiar regression view

$$\mathbf{y}_t = \mathbf{c} + \Phi \mathbf{X}_t + \mathbf{B} \mathbf{x}_t + \boldsymbol{\varepsilon}_t. \quad (3.3)$$

VAR/VARX models capture both auto-dependence and cross-series effects but introduce C^2p lag parameters (plus exogenous terms), creating a tension between expressivity and sample efficiency that motivates pooling and shrinkage below.

3.3 Unit Heterogeneity and Pooling (Panels)

Consider a panel $\{\mathbf{y}_{i,t} \in \mathbb{R}^C : i \in [N], t \in \mathcal{T}_i\}$ where entities share broad dynamics yet differ in baselines and possibly interactions. A central modeling choice is how much to share information across entities.

3.3.1 Fixed Effects (Within-Entity Intercepts)

A panel VAR(p) with entity-specific intercepts is

$$\mathbf{y}_{i,t} = \boldsymbol{\alpha}_i + \sum_{k=1}^p \Phi_k \mathbf{y}_{i,t-k} + \mathbf{B} \mathbf{x}_{i,t} + \boldsymbol{\varepsilon}_{i,t}. \quad (3.4)$$

Within transformation (demeaning) removes $\boldsymbol{\alpha}_i$, controlling for persistent baseline differences. It is effective with reasonably long time series, though one must take care in very short panels where lags and demeaning interact.

3.3.2 Random Effects and Partial Pooling

Random effects treat unit-specific parameters as draws from a population distribution, inducing partial pooling. For example, intercepts

$$\alpha_i \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Psi}) \quad (3.5)$$

shrink entity baselines toward $\boldsymbol{\mu}$, with greater shrinkage for entities with less information. The idea extends to slope parameters, enabling structured sharing while preserving entity-specific deviations.

3.3.3 Pooling Taxonomy and a Block-Diagonal Example

No pooling estimates each entity (or group) independently; in a multivariate system, this can be encoded via block-diagonal lag matrices:

$$\boldsymbol{\Phi}_k = \begin{bmatrix} \boldsymbol{\Phi}_k^{(1)} & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & \boldsymbol{\Phi}_k^{(2)} & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & \boldsymbol{\Phi}_k^{(G)} \end{bmatrix}. \quad (3.6)$$

Complete pooling forces a common parameter set across all entities, so they share the same parameters:

$$\mathbf{y}_{i,t} = \sum_{k=1}^p \boldsymbol{\Phi}_k \mathbf{y}_{i,t-k} + \boldsymbol{\varepsilon}_{i,t}, \quad \text{same } \boldsymbol{\Phi}_k \text{ for all } i. \quad (3.7)$$

Partial pooling lies between: parameters vary by entity but are regularized toward a shared structure, improving generalization when some entities are data-poor.

3.3.4 Mixed-Effects VAR (Unified Template)

A compact formulation combines shared dynamics and entity-specific deviations:

$$\mathbf{y}_{i,t} = \boldsymbol{\alpha}_i + \sum_{k=1}^p (\boldsymbol{\Phi}_k + \mathbf{B}_{i,k}) \mathbf{y}_{i,t-k} + \mathbf{B} \mathbf{x}_{i,t} + \boldsymbol{\varepsilon}_{i,t}, \quad \text{vec}(\mathbf{B}_{i,k}) \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Omega}), \quad (3.8)$$

where $\mathbf{B}_{i,k}$ encode unit-level deviations that are shrunk toward zero. This hierarchical view formalizes “share when helpful, specialize when necessary.”

3.4 Bayesian VAR in High Dimensions

3.4.1 Why Shrinkage?

In a VAR(p), the lag parameters alone number C^2p . With modest C and p , ordinary least squares can overfit and yield unstable forecasts. Bayesian approaches address this by placing informative priors over coefficients, shrinking them toward structured baselines to trade a small amount of bias for a large reduction in variance. This enables “large” BVARs that would otherwise be infeasible (3).

3.4.2 Structured Shrinkage: The Minnesota Prior

A canonical prior encodes three beliefs: (i) higher lags matter less, (ii) cross-variable effects are typically smaller than own-lags, and (iii) own first lags may be near a random-walk benchmark. Let $\Phi_{ij}^{(k)}$ denote the effect of variable j on variable i at lag k . A convenient element-wise Gaussian prior is

$$\Phi_{ij}^{(k)} \sim \mathcal{N}(m_{ij}^{(k)}, v_{ij}^{(k)}), \quad m_{ij}^{(k)} = \begin{cases} 1 & \text{if } i = j, k = 1, \\ 0 & \text{otherwise,} \end{cases} \quad (3.9)$$

with variances

$$v_{ij}^{(k)} = \frac{\lambda^2}{k^d} \times \underbrace{\begin{cases} \kappa_{\text{own}}^2 & i = j, \\ \kappa_{\text{cross}}^2 & i \neq j, \end{cases}}_{\text{own vs. cross shrinkage}} \times \underbrace{\frac{\sigma_i^2}{\sigma_j^2}}_{\text{scale by innovation variances}}. \quad (3.10)$$

Here $\lambda > 0$ controls overall tightness, $d > 0$ controls lag decay, and $(\kappa_{\text{own}}, \kappa_{\text{cross}})$ set the relative shrinkage between own and cross coefficients (typically $\kappa_{\text{cross}} < \kappa_{\text{own}}$). The scale ratio σ_i^2/σ_j^2 (from univariate residuals) adjusts for differences in measurement units. This Minnesota structure delivers stable forecasts in sizable systems and is widely used in practice (2; 3).

Hyperparameter selection. Tightness λ , lag decay d , and the own/cross ratio can be set by (i) empirical Bayes (maximize marginal likelihood on a training slice), (ii) simple grid search with time-series cross-validation, or (iii) rules-of-thumb tailored to horizon and panel size. The key is to calibrate shrinkage strength to the available information and the degree of cross-series signal.

3.4.3 Alternative Shrinkage Priors (Briefly)

Beyond Minnesota’s structured shrinkage, one may use generic shrinkage (ridge: isotropic Gaussian; lasso: Laplace promoting sparsity) or global–local priors (e.g., horseshoe) to adaptively retain a small number of strong coefficients while shrinking the rest. These choices trade interpretability and computational cost against flexibility; we revisit analogous ideas in the deep model’s selective mixing in Chapter 5.

3.5 Innovation Covariance Σ (Forecasting View)

The innovation covariance Σ governs cross-sectional co-movement and the dispersion of forecast distributions. Common choices include (i) diagonal Σ for simplicity, (ii) full but shrunk covariance (e.g., toward diagonal) for stability, or (iii) low-rank/factor structures to capture shared shocks parsimoniously. In forecasting applications, the choice impacts both point forecasts (through multistep propagation) and uncertainty quantification. As with coefficient shrinkage, the guiding principle is to encode reasonable structure without exhausting effective sample size.

3.6 Summary and Link to the Proposed Model

This chapter established a compact toolkit for multivariate time series: VAR/VARX for dynamic cross-series dependence, pooling mechanisms for panel heterogeneity, and Bayesian shrinkage, especially Minnesota, to stabilize high-dimensional estimation (2; 3). In Chapter 5, these principles are transposed into a deep-learning setting: structured partial pooling to learn cross-channel effects only when supported by data, and hierarchy-aware training to respect aggregation structure, all while retaining the scalability benefits of modern sequence encoders.

Chapter 4

Deep Neural Networks for Time Series

The application of deep learning architectures has significantly augmented the capacity of statistical learning to model latent factors and complex nonlinear relationships. Consequently, deep neural networks have been increasingly explored for time series analysis, where they offer substantial improvements over classical statistical methods in capturing intricate dynamics and long-range temporal dependencies (17).

This chapter surveys the evolution of key neural architectures for time series modeling, beginning with Recurrent Neural Networks (RNNs) and their solution to the short-term memory problem, the Long Short-Term Memory (LSTM) architecture. We then transition to the attention-based Transformer framework and conclude with an in-depth analysis of a state-of-the-art adaptation for time series, the PatchTST model.

4.1 Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNNs) were among the first architectures specifically designed to operate on sequential data (17). Unlike feedforward networks, which process inputs independently, RNNs possess a form of memory that allows information to persist. This is achieved through a feedback loop where the output from one step is fed back as input to the current step.

For a time series $\mathbf{x} = (x_1, x_2, \dots, x_T)$, the core of an RNN is the hidden state, $\mathbf{h}_t$, which is updated at each timestep t . This state acts as a compressed representation of the sequence's history up to that point. The update mechanism is governed by the following recurrence relation:

A standard RNN processes an input sequence $\{\mathbf{x}_t\}_{t=1}^T$ via

$$\mathbf{h}_t = \sigma_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h), \quad (4.1)$$

$$\mathbf{y}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y, \quad (4.2)$$

where $\mathbf{h}_{t-1}$ is the hidden state from the previous timestep, $\mathbf{x}_t$ is the input at the current timestep, $\mathbf{W}_{hh}$ and $\mathbf{W}_{xh}$ are weight matrices, $\mathbf{b}_h$ is a bias vector, and σ_h is a non-linear activation function, typically a hyperbolic tangent ($\tanh$)(17).

This recurrent nature allows RNNs to model temporal dependencies, but in practice, they struggle to learn long-range patterns due to the problem of vanishing and exploding gradient during backpropagation through time. Here, the influence of past inputs either diminishes or grows exponentially over time, making learning long-range dependencies difficult (17).

4.2 Long Short-Term Memory (LSTM) Networks

To address the limitations of simple RNNs, the Long Short-Term Memory (LSTM) architecture was introduced (18). LSTMs are a sophisticated type of RNN that incorporate a more complex cell structure designed to regulate the flow of information. The key innovation is the cell state, $\mathbf{c}_t$, which acts as a long-term memory conveyor belt, allowing information to pass through the network largely unchanged.

The flow of information into and out of this cell state is controlled by three specialized mechanisms called “gates”: the forget gate, the input gate, and the output gate. At each timestep t , these gates, which are composed of a sigmoid neural network layer and a pointwise multiplication operation, decide which information to discard from the cell state, which new information to store, and what to output. The update equations are as follows:

$$f_t = \sigma(\mathbf{W}_f\mathbf{x}_t + \mathbf{U}_f\mathbf{h}_{t-1} + \mathbf{b}_f) \quad (\text{Forget Gate}) \quad (4.3)$$

$$i_t = \sigma(\mathbf{W}_i\mathbf{x}_t + \mathbf{U}_i\mathbf{h}_{t-1} + \mathbf{b}_i) \quad (\text{Input Gate}) \quad (4.4)$$

$$o_t = \sigma(\mathbf{W}_o\mathbf{x}_t + \mathbf{U}_o\mathbf{h}_{t-1} + \mathbf{b}_o) \quad (\text{Output Gate}) \quad (4.5)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c\mathbf{x}_t + \mathbf{U}_c\mathbf{h}_{t-1} + \mathbf{b}_c) \quad (\text{Candidate Cell State}) \quad (4.6)$$

$$\mathbf{c}_t = f_t \odot \mathbf{c}_{t-1} + i_t \odot \tilde{\mathbf{c}}_t \quad (\text{Cell State Update}) \quad (4.7)$$

$$\mathbf{h}_t = o_t \odot \tanh(\mathbf{c}_t) \quad (\text{Hidden State Update}) \quad (4.8)$$

where σ denotes the sigmoid, $\odot$ is element-wise multiplication, and $W_{\{f,i,o,c\}}, U_{\{f,i,o,c\}}, b_{\{f,i,o,c\}}$ are learned parameters. This gating mechanism enables LSTMs to selectively remember or

forget information over long time intervals, making them significantly more effective than standard RNNs at capturing long-range dependencies. By allowing the cell state c_t to preserve gradients, the mechanism overcomes the vanishing gradient problem and ensures that long-term dependencies are retained (18).

4.3 The Transformer Framework

The Transformer architecture, introduced by (19), marks a paradigm shift in sequential modeling as it replaces recurrence with self-attention, processing all positions in parallel. It relies almost entirely on a mechanism called self-attention, which allows it to weigh the importance of different parts of the input sequence when processing a specific position. Instead of processing data in order, self-attention allows the model to look at all other positions in the sequence simultaneously.

The core of self-attention lies in three learnable vectors derived from each input token: the Query $\mathbf{Q}$, Key $\mathbf{K}$, and Value $\mathbf{V}$. The Query represents the current token's focus, the Key represents the content of other tokens, and the Value contains the information to be aggregated. The attention score is computed by taking the dot product of the Query of the current token with the Keys of all other tokens. This score determines how much focus to place on other parts of the input. The final output for a token is a weighted sum of the Values, where the weights are the attention scores. The specific formulation used is Scaled Dot-Product Attention:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V} \quad (\text{Scaled Dot-Product Attention}) \quad (4.9)$$

where d_k is the dimension of the key vectors, and $\mathbf{Q} = \mathbf{X}\mathbf{W}^Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}^K$, and $\mathbf{V} = \mathbf{X}\mathbf{W}^V$ are learned linear projections of the input $\mathbf{X}$.

The Transformer extends this to multi-head self-attention, which allows the model to jointly attend to information from different representation subspaces:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O \quad (\text{Multi-Head Output}) \quad (4.10)$$

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V) \quad (\text{Individual Head}) \quad (4.11)$$

While revolutionary for natural language processing, the vanilla Transformer has two major drawbacks for time series forecasting. First, the self-attention mechanism has a com-

putational and memory complexity of $O(T^2)$ with respect to the sequence length T , making it prohibitively expensive for long time series. Second, it is a permutation-invariant model; it does not inherently recognize the temporal order of the data, which is a critical feature of time series. These limitations necessitate specialized adaptations to make the Transformer architecture viable for forecasting tasks.

4.4 Patch Time Series Transformer (PatchTST)

The Patch Time Series Transformer (PatchTST) was proposed to address limitations of applying Transformers to long-horizon time series forecasting (7). The key idea is to treat a time series not as individual points but as a sequence of subseries-level patches, analogous to words in a sentence. This reduces sequence length, focuses attention on semantically meaningful local patterns (e.g., seasonal fragments), and makes computation tractable.

Given C univariate channels and a recent context $\mathbf{X}_{\text{raw}} \in \mathbb{R}^{C \times L}$, the goal is to produce a forecast $\hat{\mathbf{Y}} \in \mathbb{R}^{C \times H}$ for horizon H via the pipeline outlined in figure 4.1.

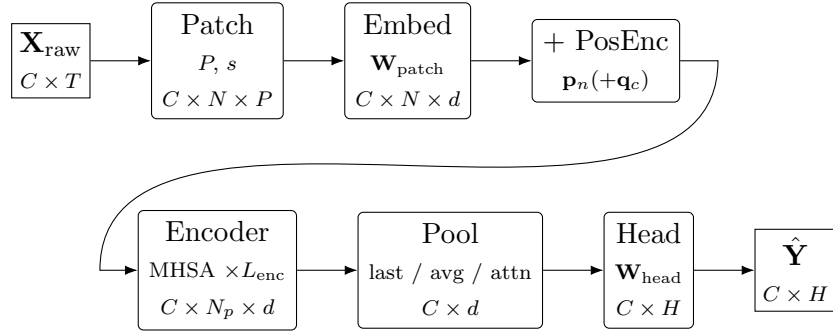


Figure 4.1: PatchTST pipeline: Patch $\rightarrow$ Embed + PosEnc $\rightarrow$ Encoder (MHSA, L layers) $\rightarrow$ Pool $\rightarrow$ Head. Tensors shown with shapes.

Throughout, B denotes batch size, d the embedding dimension, L_{enc} the number of Transformer layers, and m the number of attention heads (we write m to avoid clashing with the forecast horizon H).

4.4.1 Patch Extraction

Choose patch length P and stride s . The number of patches is

$$N_p = \left\lfloor \frac{L - P}{s} \right\rfloor + 1. \quad (4.12)$$

For channel c and patch index n , the n -th patch is the vector $\mathbf{x}_{c,n} \in \mathbb{R}^P$. Stacking these creates the full tensor of patches:

$$\underbrace{\mathbf{X} \in \mathbb{R}^{C \times N_p \times P}}_{\text{tokens per channel}}, \quad \underbrace{P}_{\text{token length}}, \quad \underbrace{s}_{\text{stride}}. \quad (4.13)$$

4.4.2 Patch Embedding and Positional Encodings

Each length- P token is linearly embedded to $\mathbb{R}^d$:

$$\mathbf{e}_{c,n} = \mathbf{x}_{c,n} \mathbf{W}_{\text{patch}}^T + \mathbf{b}_{\text{patch}} \in \mathbb{R}^d, \quad \underbrace{\mathbf{W}_{\text{patch}} \in \mathbb{R}^{d \times P}, \mathbf{b}_{\text{patch}} \in \mathbb{R}^d}_{\text{shared across channels}}. \quad (4.14)$$

To encode temporal order, a position vector $\mathbf{p}_n \in \mathbb{R}^d$ (sinusoidal or learned) is added:

$$\tilde{\mathbf{e}}_{c,n} = \mathbf{e}_{c,n} + \mathbf{p}_n, \quad \underbrace{n = 1, \dots, N_p}_{\text{temporal order}}. \quad (4.15)$$

Optionally, a learnable channel-ID embedding $\mathbf{q}_c \in \mathbb{R}^d$ can also be added: $\tilde{\mathbf{e}}_{c,n} \leftarrow \tilde{\mathbf{e}}_{c,n} + \mathbf{q}_c$.

4.4.3 Transformer Encoder

PatchTST processes each channel's token sequence independently with shared encoder weights. The encoder is composed of L_{enc} layers of bidirectional Multi-Head Self-Attention (MHSA).

$$\{\tilde{\mathbf{e}}_{c,n}\}_{n=1}^{N_p} \mapsto \{\mathbf{h}_{c,n}\}_{n=1}^{N_p} = \mathcal{T}_\theta(\{\tilde{\mathbf{e}}_{c,n}\}_{n=1}^{N_p}), \quad \underbrace{\theta \text{ shared for all } c}_{\text{parameter sharing}}. \quad (4.16)$$

Because the entire lookback window is historical data, attention is bidirectional, allowing every patch to attend to every other patch to capture the full context.

For a fixed channel (index c dropped), we form queries, keys, and values from the input embeddings $\tilde{\mathbf{E}} \in \mathbb{R}^{N_p \times d}$:

$$\mathbf{Q} = \tilde{\mathbf{E}} \mathbf{W}_Q, \quad \mathbf{K} = \tilde{\mathbf{E}} \mathbf{W}_K, \quad \mathbf{V} = \tilde{\mathbf{E}} \mathbf{W}_V, \quad \mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d \times d_k}. \quad (4.17)$$

Attention scores are computed and applied to the values:

$$\text{Attn} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right). \quad (4.18)$$

The token representations are then updated via residual connections and a position-wise

feed-forward network (pre/post-norm omitted for brevity):

$$\mathbf{U} = \text{Attn } \mathbf{V} \mathbf{W}_O, \quad \mathbf{H} = \tilde{\mathbf{E}} + \mathbf{U} + \text{MLP}(\tilde{\mathbf{E}} + \mathbf{U}), \quad (4.19)$$

$$\underbrace{\mathbf{W}_O \in \mathbb{R}^{d_k m \times d}}_{\text{projection}}, \quad \underbrace{\text{MLP}(\mathbf{x}) = \mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x})}_{\text{positionwise feed-forward}}. \quad (4.20)$$

With m heads, this is computed in parallel over different subspaces of the embedding dimension. The computational cost of multi-head self-attention scales quadratically with the patch sequence length. For a single channel, the encoder requires $O(N_p^2 d)$ time and $O(N_p^2)$ memory, where N_p is the number of patches and d the embedding dimension. Aggregating across C channels, the total complexity is $O(CN_p^2 d)$.

4.4.4 Feature Representation

After L_{enc} layers, form a fixed-size vector per channel by vectorizing the output:

$$\mathbf{z}_c = \text{vec}([\mathbf{h}_{c,1}, \dots, \mathbf{h}_{c,N_p}]) \in \mathbb{R}^{N_p d}. \quad (4.21)$$

4.4.5 Prediction Head and Training Objective

$$\hat{\mathbf{y}}_c = \mathbf{z}_c^\top \mathbf{W}_{\text{head}} + \mathbf{b}_{\text{head}} \in \mathbb{R}^H, \quad \mathbf{W}_{\text{head}} \in \mathbb{R}^{N_p d \times H}, \quad \mathbf{b}_{\text{head}} \in \mathbb{R}^H. \quad (4.22)$$

The model is trained end-to-end by minimizing a loss function, typically the Mean Squared Error (MSE), between the forecasts $\hat{\mathbf{Y}}$ and actuals $\mathbf{Y}$:

$$\mathcal{L}_{\text{pred}} = \frac{1}{CH} \sum_{c=1}^C \sum_{r=1}^H (\hat{Y}_{c,r} - Y_{c,r})^2. \quad (4.23)$$

4.4.6 Why Patches Help

Patching provides several advantages: (i) it reduces the effective sequence length for the Transformer from L to N_p , making attention computationally feasible on long series; (ii) it allows the embedding layer $\mathbf{W}_{\text{patch}}$ to learn local features within patches before the attention mechanism aggregates these features globally; and (iii) it imposes a useful inductive bias by separating local (within-patch) and global (across-patch) information processing.

4.4.7 Model Rationale and Limitations

By converting the time series into patch tokens, PatchTST lowers computational burden and directs capacity toward higher-level temporal motifs, while channel-wise processing with shared weights improves efficiency and generalization. However, strict channel independence prevents explicit learning of cross-channel dependencies. In settings with interacting variables (e.g., cannibalization across product lines), this assumption is suboptimal, motivating extensions that introduce controlled cross-channel mixing within the PatchTST framework. This limitation motivates the selectively regularized, hierarchy-aware cross-channel mixing introduced in Chapter 5.

4.4.8 Architectural Summary and Schematic

The vanilla PatchTST model maps a historical context window $\mathbf{X}_{\text{raw}} \in \mathbb{R}^{C \times L}$ to a horizon- H forecast $\hat{\mathbf{Y}} \in \mathbb{R}^{C \times H}$ through a multi-step process. This involves: (i) partitioning each channel’s time series into overlapping, length- P patches (tokens) with a stride s ; (ii) linearly embedding these patch-tokens and adding positional encodings; (iii) passing each channel’s sequence of embedded tokens through a shared-weight Transformer encoder, which uses bidirectional Multi-Head Self-Attention (MHSA); (iv) creating a fixed-size summary vector for each channel from the encoder’s output; and finally, (v) applying a per-channel linear head to this summary vector to produce the forecast. In this channel-independent design, the encoder and head weights are shared across all channels, and no explicit cross-channel mixing is performed.

Pipeline Identities.

$$\begin{aligned}
\text{Tokens: } & \mathbf{x}_{c,n} \in \mathbb{R}^P \quad (\text{for } n = 1, \dots, N_p), \\
\text{Embed: } & \mathbf{e}_{c,n} = \mathbf{x}_{c,n} \mathbf{W}_{\text{patch}}^T + \mathbf{b}_{\text{patch}} \in \mathbb{R}^d, \\
\text{Positional Encoding: } & \tilde{\mathbf{e}}_{c,n} = \mathbf{e}_{c,n} + \mathbf{p}_n(+\mathbf{q}_c), \\
\text{Encoder: } & \{\tilde{\mathbf{e}}_{c,n}\}_{n=1}^{N_p} \mapsto \{\mathbf{h}_{c,n}\}_{n=1}^{N_p} \quad \text{via MHSA over } L \text{ layers}, \\
\text{Representation: } & \mathbf{z}_c = \text{Flatten}(\{\mathbf{h}_{c,n}\}_{n=1}^{N_p}) \in \mathbb{R}^{N_p \times d}, \\
\text{Head: } & \hat{\mathbf{y}}_c = \mathbf{z}_c^\top \mathbf{W}_{\text{head}} + \mathbf{b}_{\text{head}} \in \mathbb{R}^H.
\end{aligned}$$

Compact Composition.

$$\hat{\mathbf{Y}} = \underbrace{\text{Head}}_{\text{per-channel linear}} \left(\underbrace{\text{Represent}}_{\text{Flatten/Last}} \left(\underbrace{\mathcal{T}_\theta}_{\text{MHSA, } L \text{ layers}} \left(\underbrace{\text{Embed}}_{\mathbf{W}_{\text{patch}}} \left(\underbrace{\text{Patch}}_{P,s}(\mathbf{X}_{\text{raw}}) \right) + \underbrace{\text{PosEnc}}_{\mathbf{p}_n(+\mathbf{q}_c)} \right) \right) \right). \quad (4.24)$$

Chapter 5

Proposed Model: Hierarchical Bayesian PatchTST

The vanilla PatchTST architecture achieves high efficiency and strong generalization by processing each time series channel independently with shared weights. This principle of channel-independence, however, makes it structurally incapable of capturing crucial cross-channel dynamics. In many real-world systems, such as retail panels with interacting products or financial markets with correlated assets, these dependencies are central to the underlying data-generating process.

Conversely, architectures with full cross-channel mixing, like those using dense forecasting heads or cross-channel attention, are highly expressive but scale quadratically in the number of series ($O(C^2)$). This makes them computationally and statistically burdensome in high-dimensional settings, often leading to overfitting. The central challenge, therefore, is to balance scalability with the capacity to learn meaningful cross-series interactions.

This chapter introduces the Hierarchical Bayesian PatchTST, a model that resolves this trade-off. It employs a factorized architecture where forecasts for a large number of bottom-level series are composed from learned representations of their underlying attributes. We inject static covariate information using feature-wise modulation and introduce structured Bayesian priors, including Minnesota-style shrinkage, to guide learning. This approach, combined with a weighted hierarchical training objective supervising latent leaf-level series, yields forecasts that are coherent, robust, and scalable to high-dimensional hierarchical systems.

5.1 Core Architecture: A Factorized Approach to Forecasting

The model’s architecture is designed to manage the complexity of a high-dimensional system by learning disentangled, factorized representations and composing them to generate forecasts. Rather than modeling all series jointly, it learns distinct temporal patterns for key entity groups (e.g., products, stores, areas) and combines them through a low-rank interaction layer.

5.1.1 Unified Hierarchical Structure

Instead of partitioning the panel into independent hierarchies, the model adopts a unified, bottom-up structure. It begins by identifying the most granular “leaf” level of the system, which in this case comprises the 96 unique time series representing each individual product’s sales within each specific retail store or e-commerce area (e.g., **P1-StoreA**, **P1-AreaX**, etc.). All higher-level time series, such as total sales for a product, a category, a channel, or the grand total, are treated as aggregations derived by summing the appropriate leaf series. This structure allows the model to learn complex interactions across the entire system from the ground up.

5.1.2 Bayesian Front-End: Structured Feature Extraction

To maintain scalability and impose economically motivated inductive biases, the model processes each factor group (Products, Stores, Areas) through a Bayesian front-end designed to regularize the initial representations instead of processing all independent series in a single, large panel.

Bayesian Patch Embedding with Temporal Decay

After partitioning each time series into patches, we treat the patch embedding weights $\mathbf{W}_{\text{patch}}$ as random variables. We place a zero-mean Gaussian prior on these weights, but with a variance structure that encodes a temporal inductive bias: the influence of older observations within a patch is shrunk more strongly than that of more recent observations. For a weight $W_{d,p}$ corresponding to the p -th timestep within a patch (where $p = 1$ is the oldest), the prior variance σ_p^2 is structured as:

$$W_{d,p} \sim \mathcal{N}(0, \sigma_p^2), \quad \sigma_p^2 \propto (L - p + 1)^{-2\alpha_{\text{patch}}} \quad (5.1)$$

Here, L is the patch length. This prior on the embedding layer itself encourages the model to prioritize more recent information when creating its initial token representations.

Factor-Group Mixing with Minnesota-Style Priors

To allow for controlled information sharing within each factor group before the main encoder, we introduce a learnable channel-mixing matrix for each group: $\mathbf{A}_p$ for products, $\mathbf{A}_s$ for stores, and $\mathbf{A}_a$ for areas. After the Bayesian patch embedding, each matrix creates a new set of tokens by linearly combining the embeddings across all series within its respective group. For the product group, this is:

$$\tilde{\mathbf{e}}_{p,n} = \sum_{j=1}^P (\mathbf{A}_p)_{p,j} \mathbf{e}_{j,n}. \quad (5.2)$$

We place a Minnesota-style prior on these mixer matrices to favor self-dependence, a core principle for stabilizing time series models. The prior on each element A_{ij} is a zero-mean Gaussian whose variance is larger for diagonal elements (an entity’s own influence, $i = j$) and smaller for off-diagonal elements (cross-entity influence, $i \neq j$). This is governed by hyperparameters $\lambda_{\text{own}} > \lambda_{\text{cross}}$:

$$A_{ij} \sim \mathcal{N}(0, \sigma_{ij}^2), \quad \sigma_{ij}^2 = \lambda_{\text{own}}^2 \mathbf{1}[i = j] + \lambda_{\text{cross}}^2 \mathbf{1}[i \neq j]. \quad (5.3)$$

This prior gently shrinks spurious cross-channel mixing at the input stage, encouraging the model to build its initial representations primarily from a series’ own history unless strong cross-series patterns are present in the data, thus avoiding spurious leakage across entities.

5.1.3 The Core Encoder

The sequences of regularized, pre-mixed, and positionally encoded tokens for each of the three factor groups are then processed by a standard, shared-weight PatchTST encoder. This encoder, composed of multiple layers of multi-head self-attention, is responsible for learning complex temporal patterns and relationships among the patches, outputting rich, fixed-dimension embeddings for each product (Z_p), store (Z_s), and area (Z_a).

5.1.4 Static Covariate Injection

To make the learned representations aware of the unique characteristics of each series, we inject static metadata (e.g., product category, store size, region) using two complementary mechanisms:

1. FiLM (Feature-wise Linear Modulation): After the encoder, a small neural network maps the one-hot encoded covariates of each entity to a pair of vectors: a scaling vector γ and a shifting vector β . These modulate the encoded representations Z , allowing the model to finely tune the learned temporal patterns based on static attributes. FiLM scales and shifts $(\gamma, \beta) \in \mathbb{R}^d$, applied feature-wise:

$$\tilde{Z}_{i,\cdot} = \gamma_i \odot Z_{i,\cdot} + \beta_i, \quad \gamma_i = 1 + \eta \cdot \tanh(W_\gamma x_i), \quad \beta_i = W_\beta x_i, \quad (5.4)$$

where x_i is the one-hot metadata vector, W_γ, W_β are learned, and $0 < \eta \ll 1$ gates the scale.

2. Horizon-wise Bias Injection: A second small network maps the static covariates of each entity i to a unique, forecast-horizon-length bias vector, or curve, denoted as $b_i \in \mathbb{R}^H$. This vector, generated by a tiny MLP (the bias bottleneck), is then mean-centered and scaled by a learnable gate coefficient $\lambda_b \in (0, 1)$ before being added to the final forecast:

$$\tilde{b}_i = \lambda_b \cdot \left(b_i - \frac{1}{H} \sum_{h=1}^H b_{i,h} \right) \quad (5.5)$$

This mechanism allows the model to learn entity-specific level shifts or trends that are not captured by the main dynamic representations.

FiLM injects entity semantics directly into features, while the bias curves provide low-complexity shape priors over the horizon (e.g., store-size or regional trends), both without inflating the temporal encoder.

5.1.5 Low-Rank Forecast Composition

The final leaf-level forecasts are generated not by a dense mixing head but through a highly efficient low-rank composition. The modulated representations for products, stores, and areas are projected into low-rank factor matrices. The forecasts are then constructed via a tensor product (implemented with `einsum`), which models the interaction between these

factors. For example, the retail forecasts $Y_r \in \mathbb{R}^{P \times S \times H}$ are composed as:

$$\hat{Y}_r = \text{einsum}('prh, srh \rightarrow psh', \mathbf{A}_p, \mathbf{S}_s) + \text{biases}. \quad (5.6)$$

Here, $\mathbf{A}_p$ and $\mathbf{S}_s$ are the low-rank representations for products and stores over the forecast horizon H . This factorization serves as a powerful structural prior, assuming that the behavior of a leaf series (e.g., sales of product p in store s) is primarily driven by an interaction between the general pattern for product p and the general pattern for store s . This avoids learning $P \times S$ independent forecast heads, dramatically reducing parameters and improving statistical efficiency.

5.2 Hierarchical Coherence and Inference

The model is trained within a probabilistic framework that encourages its forecasts to be consistent across all levels of the hierarchy.

5.2.1 Soft Reconciliation via a Weighted Hierarchical Likelihood

To ensure hierarchical coherence, we employ a soft reconciliation mechanism. The model’s core architecture composes forecasts for all the most granular, leaf-level series (e.g., ‘P1-StoreA’). However, these leaf series are treated as latent variables that are not directly observed during training. Instead, the model is supervised using only the observed, higher-level aggregates.

The forecasts for these parent series are derived by aggregating the latent leaf forecasts according to the known summation structure, which can be expressed formally with a summing matrix S :

$$\hat{Y}^{(p)} = S \hat{Y}^{(\ell)} \quad (5.7)$$

The training objective is constructed from a set of weighted Gaussian likelihood terms, one for each group of parent series g (e.g., channels, categories, grand total). This creates a single objective function that softly reconciles the hierarchy, as parent-level supervision implicitly guides the latent leaf forecasts to sum correctly. The full weighted log-likelihood is:

$$\mathcal{L}_{\text{lik}} = \sum_{g \in \mathcal{G}} \omega_g \sum_{i \in \mathcal{I}_g} \sum_{h=1}^H \log \mathcal{N}(Y_{i,h}^{(p)} \mid \hat{Y}_{i,h}^{(p)}, \sigma_g^2) \quad (5.8)$$

The weights ω_g are crucial, as they allow us to prioritize coherence at more important or reliable aggregation levels. In this implementation, we place the highest weight on channel

totals, followed by the grand total, and then the lower levels. This forces the model to learn the most stable patterns first.

This structure facilitates a powerful form of inductive transfer. By focusing the learning on stable, information-rich aggregates like channel totals, the model captures robust signals. Through the deterministic summation (S) and the shared low-rank factors in the architecture, these signals are automatically propagated down to properly shape all S latent leaf-level forecasts, all within a single, unified training process.

5.2.2 Probabilistic Inference via SVI

The true posterior distribution over the model’s Bayesian parameters (e.g., patch embeddings, forecast composition weights) is intractable. We therefore use Stochastic Variational Inference (SVI) to approximate it. We introduce a simpler, parameterized variational distribution $q_\phi(\mathbf{W})$ and optimize its parameters ϕ by maximizing the Evidence Lower Bound (ELBO):

$$\mathcal{L}_{\text{ELBO}}(\phi) = \underbrace{\mathbb{E}_{q_\phi(\mathbf{W})} [\log p(\text{Data} \mid \mathbf{W})]}_{\text{Expected Log Likelihood}} - \underbrace{\text{KL}(q_\phi(\mathbf{W}) \parallel p(\mathbf{W}))}_{\text{Complexity Penalty}}. \quad (5.9)$$

The first term encourages the model to fit the observed data across the entire hierarchy (respecting the likelihood weights), while the second term regularizes the model by ensuring the approximate posterior stays close to our structured priors. This unified objective allows for end-to-end training, balancing data fidelity with the inductive biases of the factorized architecture.

5.3 Model Complexity and Scalability

The architectural choices of the model place its complexity and scalability favorably between common extremes. A classical Vector Autoregression (VAR) model, for instance, requires fitting a dense matrix of coefficients that scales quadratically with the number of series, $O(C^2)$, making it computationally infeasible for a system of 96 series. Similarly, a PatchTST with full channel-mixing (e.g., a dense linear forecasting head) would also have parameters that scale as $O(C^2)$, posing a significant risk of overfitting in addition to the computational cost. At the other end, a vanilla channel-independent PatchTST scales linearly, $O(C)$, but is structurally unable to capture any cross-series dependencies.

This model provides a practical intermediate. By learning representations for the underlying factors (P products, S stores, A areas) and composing them via a low-rank (r) head,

the parameter complexity is dominated by the factors, scaling closer to $O((P + S + A) \cdot d \cdot r)$, which is dramatically more efficient than the $O((P \cdot (S + A))^2)$ complexity of a fully dense approach. This factorized structure acts as a form of scalable partial pooling, enabling the model to share statistical strength across series in a structured and efficient manner.

5.4 Algorithmic Specification

To bridge the gap between the conceptual architecture and the implementation, we present the model’s logic in pseudocode. We first outline the probabilistic generative process of the model itself (Algorithm 1), followed by the Stochastic Variational Inference (SVI) training procedure (Algorithm 2).

Algorithm 1 Factorized Probabilistic Generative Model

- 1: **Input:** windows $\mathbf{X}_p, \mathbf{X}_s, \mathbf{X}_a$; fixed covariates $\mathbf{C}_p, \mathbf{C}_s, \mathbf{C}_a$
 - 2: **Output:** parent forecasts $\{\hat{\mathbf{Y}}_g\}_{g \in \{\text{prod, cat, chan, store, area, gt}\}}$
 - 3: **Front-end priors:** $\mathbf{W}_{\text{patch}}, \mathbf{b}_{\text{patch}} \sim \mathcal{P}_{\text{patch}}; \mathbf{A}_g \sim \mathcal{P}_{\text{mixer}}$ for $g \in \{p, s, a\}$
 - 4: **Head priors:** $\mathbf{W}_{g,f} \sim \mathcal{N}(0, \sigma_{wf}^2 \mathbf{I})$ for $g \in \{p, s, a\}; \mathbf{b}_p, \mathbf{b}_s, \mathbf{b}_a, \mathbf{b}_0 \sim \mathcal{N}(0, \cdot); \lambda_p, \lambda_s, \lambda_a \in (0, 1)$
 - 5: **Encode:** $\mathbf{Z}_g \leftarrow \text{Encoder}(\mathbf{X}_g; \mathbf{W}_{\text{patch}}, \mathbf{b}_{\text{patch}}, \mathbf{A}_g)$ for $g \in \{p, s, a\}$
 - 6: **Covariates:** $(\gamma_g, \beta_g) \leftarrow \text{FiLM}(\mathbf{C}_g); \mathbf{Z}'_g = \mathbf{Z}_g \odot \gamma_g + \beta_g; \mathbf{b}_g \leftarrow \mathbf{b}_g + \lambda_g \text{BiasHead}(\mathbf{C}_g)$
 - 7: **Low-rank maps (rank R , decay $d_h \propto h^{-\alpha}$):**
 - 8: $\mathbf{A}_p^{(r,h)} \leftarrow \text{reshape}(\mathbf{Z}'_p \mathbf{W}_{p,f}) \odot \mathbf{d}; \mathbf{S}_s^{(r,h)} \leftarrow \text{reshape}(\mathbf{Z}'_s \mathbf{W}_{s,f}) \odot \mathbf{d}; \mathbf{E}_a^{(r,h)} \leftarrow \text{reshape}(\mathbf{Z}'_a \mathbf{W}_{a,f}) \odot \mathbf{d}$
 - 9: **Leaves (unobserved):**
 - 10: $\hat{\mathbf{Y}}_{\text{retail}}^{(\ell)} = \sum_r \mathbf{A}_p^{(r,\cdot)} \otimes \mathbf{S}_s^{(r,\cdot)} + \mathbf{b}_p \oplus \mathbf{b}_s \oplus \mathbf{b}_0$
 - 11: $\hat{\mathbf{Y}}_{\text{ecomm}}^{(\ell)} = \sum_r \mathbf{A}_p^{(r,\cdot)} \otimes \mathbf{E}_a^{(r,\cdot)} + \mathbf{b}_p \oplus \mathbf{b}_a \oplus \mathbf{b}_0$
 - 12: **Aggregate to parents:**
 - 13: $\hat{\mathbf{Y}}_{\text{prod}} = \sum_s \hat{\mathbf{Y}}_{\text{retail}}^{(\ell)} + \sum_a \hat{\mathbf{Y}}_{\text{ecomm}}^{(\ell)}; \hat{\mathbf{Y}}_{\text{cat}} = \mathbf{S}_{\text{cat} \leftarrow \text{prod}} \hat{\mathbf{Y}}_{\text{prod}}$
 - 14: $\hat{\mathbf{Y}}_{\text{chan}} = \begin{bmatrix} \sum_{p,s} \hat{\mathbf{Y}}_{\text{retail}}^{(\ell)} \\ \sum_{p,a} \hat{\mathbf{Y}}_{\text{ecomm}}^{(\ell)} \end{bmatrix}; \hat{\mathbf{Y}}_{\text{store}} = \sum_p \hat{\mathbf{Y}}_{\text{retail}}^{(\ell)}; \hat{\mathbf{Y}}_{\text{area}} = \sum_p \hat{\mathbf{Y}}_{\text{ecomm}}^{(\ell)}; \hat{\mathbf{Y}}_{\text{gt}} = \sum \hat{\mathbf{Y}}_{\text{chan}}$
 - 15: **Normalize:** $\tilde{\mathbf{Y}}_g = (\hat{\mathbf{Y}}_g - \mu_g) / \sigma_g \quad \forall g$
 - 16: **Likelihood (parents only, weighted):** $\mathbf{Y}_g \sim \mathcal{N}(\tilde{\mathbf{Y}}_g, \sigma_g^2 \mathbf{I})$ with weight $w_g / |g|$, where $g \in \{\text{chan, gt, area, store, cat, prod}\}$
-

Algorithm 2 SVI Training Loop (Compact)

```
1: Initialize  $\theta$  (Encoder, FiLM, BiasHead),  $\phi$  (guide); optimizer = AdamW, scheduler =  
   ExponentialLR( $\gamma = 0.95$ )  
2: for epoch = 1 to NumEpochs do  
3:   for batch ( $\mathbf{X}_p, \mathbf{X}_s, \mathbf{X}_a, \mathbf{Y}^{\text{prod}}, \mathbf{Y}^{\text{cat}}, \mathbf{Y}^{\text{chan}}, \mathbf{Y}^{\text{store}}, \mathbf{Y}^{\text{area}}, \mathbf{Y}^{\text{gt}}$ ) do  
4:     loss  $\leftarrow$  SVI.step( $\cdot$ )  $\triangleright$  parents-only ELBO with group weights  
5:   end for  
6:   Log ELBO and quick validation MAPEs (GT, Products, Categories, Channels)  
7: end for  
8: Return:  $\theta^*, \phi^*$ 
```

5.5 Model Implementation with PyTorch and Pyro

This section details the practical implementation of the model, realized using PyTorch for GPU-accelerated tensor operations and Pyro for probabilistic programming. The implementation follows the factorized architecture described previously, comprising five key stages: (i) factorized data organization, (ii) a Bayesian front-end with structured priors, (iii) static covariate injection via feature modulation, (iv) a low-rank composition layer for forecasting latent leaves, and (v) a weighted hierarchical likelihood for soft reconciliation.

Instead of treating the system as independent trees, we organize the data into three factor groups:

- Products (P): Six time series representing the total sales for each product.
- Stores (S): Time series for each retail store.
- Areas (A): Time series for each e-commerce area.

These groups are normalized independently using train-set statistics. The model is trained on sliding windows of length L from these three panels ($\mathbf{X}_p, \mathbf{X}_s, \mathbf{X}_a$) to generate forecasts for a horizon of length H .

5.5.1 The Bayesian Front-End

The front-end regularizes the initial feature extraction using a two-stage Bayesian process applied to each factor group.

Bayesian Patch Embedding with Temporal Decay

The weights $\mathbf{W}_{\text{patch}}$ and biases $\mathbf{b}_{\text{patch}}$ of the initial patch embedding layer are treated as random variables. A zero-mean Gaussian prior is placed on the weights, with a variance that shrinks the influence of older observations within a patch, prioritizing recent information. For a weight corresponding to the q -th oldest timestep within a patch of length ‘patch len’:

$$\mathbf{W}_{\text{patch}}[:, q] \sim \mathcal{N}(\mathbf{0}, (\sigma_{\text{patch}} \cdot (q + 1)^{-\alpha_{\text{patch}}})^2 \mathbf{I}). \quad (5.10)$$

Factor-Group Mixing with Minnesota-Style Priors

Before the main encoder, a learnable mixing matrix $(\mathbf{A}_p, \mathbf{A}_s, \mathbf{A}_a)$ allows for controlled information sharing within each factor group. A Minnesota-style prior is placed on each mixer, shrinking cross-entity interactions more than self-interactions ($\lambda_{\text{own}} > \lambda_{\text{cross}}$).

```
1  # Pyro: Priors for the Bayesian front-end (in the `model` function)
2  # 1. Age-based decay for patch weights
3  age = (patch_len - torch.arange(patch_len, device=device))
4  col_scale = pyro.sample("sigma_patch", ...) * (1.0 / (age ** ALPHA_PATCH))
5  W_patch = pyro.sample("W_patch", dist.Normal(0, col_scale.view(1,-1)).to_event(2))
6
7  # 2. Minnesota-style prior on the factor-group mixers
8  def mixer(name, N, sig0):
9      sigma = pyro.sample(f"sigma_{name}", ...)
10     scale = sigma * (LAMBDA1_MIX*torch.eye(N) + LAMBDA2_MIX*(1 - torch.eye(N)))
11     return pyro.sample(name, dist.Normal(0, scale).to_event(2))
12  A_p = mixer("A_p", P, ...)
13  A_s = mixer("A_s", S, ...)
14  A_a = mixer("A_a", A, ...)
```

The deterministic forward pass through the front-end and shared encoder produces feature representations for each factor: $\mathbf{Z}_p$, $\mathbf{Z}_s$, and $\mathbf{Z}_a$.

```
1  # PyTorch/Pyro: Encoder pass
2  Zp = encoder.forward_with_patch(Xp, W_patch, b_patch, A=A_p) # [B, P, d]
3  Zs = encoder.forward_with_patch(Xs, W_patch, b_patch, A=A_s) # [B, S, d]
4  Za = encoder.forward_with_patch(Xa, W_patch, b_patch, A=A_a) # [B, A, d]
```

5.5.2 Covariate Injection and Feature Modulation

After encoding, the representations $\mathbf{Z}$ are modulated using static covariates for each entity. This is handled by deterministic PyTorch modules registered with Pyro.

- FiLM (Feature-wise Linear Modulation): A small MLP maps static features to a scaling vector γ and a shifting vector β , which are applied to the encoded representations.
- Horizon-wise Bias Head: A second MLP with a small bottleneck maps static features to a horizon-length bias curve, which is mean-centered and added to the final forecast.

```
1  # PyTorch modules are registered with Pyro
2  pyro.module("film_p", film_p); pyro.module("bh_p", bh_p) # etc. for s, a
3
4  # FiLM application in the `model` function
5  gamma_p, beta_p = film_p(X_prod_t)
6  Zp = Zp * gamma_p[None, :, :] + beta_p[None, :, :]
7
8  # Gated bias application in the `model` function
9  lam_p = torch.sigmoid(pyro.param("lambda_p", ...)) # Learnable gate
10 bp_cov = bh_p(X_prod_t)
11 bp = pyro.sample("bp", ...) + lam_p * bp_cov # Add covariate bias to base bias
```

5.5.3 Low-Rank Forecast Composition

The model avoids a dense, quadratically-scaling forecasting head. Instead, it composes forecasts for the latent leaves through a low-rank factorization. The modulated features $\mathbf{Z}$ are projected into low-rank ($r = 4$) horizon-long matrices, which are then combined via `einsum`. A deterministic horizon decay ($h^{-\alpha_{\text{decay}}}$) is applied to shrink long-range forecasts.

$$\hat{Y}_r[p, s, h] = \sum_{k=1}^r \left(\langle \mathbf{z}_p^{(p)}, \mathbf{W}_{:,k,h}^{(p)} \rangle \cdot \langle \mathbf{z}_s^{(s)}, \mathbf{W}_{:,k,h}^{(s)} \rangle \right) \cdot h^{-\alpha_{\text{decay}}} + \text{biases}_{p,s,h} \quad (5.11)$$

```
1  # Pyro: Priors for the low-rank factor maps
2  sigma_wf = pyro.sample("sigma_wf", dist.LogNormal(...))
3  Wp_f = pyro.sample("Wp_f", dist.Normal(0, sigma_wf).to_event(2)) # [d, r*H]
4  Ws_f = pyro.sample("Ws_f", dist.Normal(0, sigma_wf).to_event(2)) # [d, r*H]
5  # ... and Wa_f
6
7  # PyTorch/Pyro: Low-rank composition in the `model` function
8  decay = 1.0 / torch.pow(torch.arange(1, H+1, ...), ALPHA_DECAY)
```

```

9
10 Ap = torch.einsum("bpd,dr->bpr", Zp, Wp_f).view(B, P, r, H) * decay
11 Ss = torch.einsum("bsd,dr->bsr", Zs, Ws_f).view(B, S, r, H) * decay
12
13 # Compose latent retail leaves (product-store interactions)
14 Y_r = torch.einsum("bprh,bsrh->bpsh", Ap, Ss) + ... # biases

```

5.5.4 Weighted Likelihood and Latent Variable Supervision

The model is supervised on the observed parent-level aggregates, treating the composed leaf-level forecasts ($\hat{Y}_r, \hat{Y}_e$) as latent variables. These latent forecasts are summed to produce predictions for all observed parent levels.

Crucially, the likelihood contribution of each parent group is weighted, allowing us to prioritize more stable or important aggregates (e.g., channel totals). This is implemented in Pyro using `poutine.scale`.

```

1 # Pyro: Weighted likelihood for the channel group
2 # W_PARENT is a dictionary of weights, e.g., {"CHAN": 8.0}
3 with pyro.plate("b_chan", B):
4     # The scale argument weights this likelihood's contribution to the ELBO
5     with poutine.scale(scale=(W_PARENT["CHAN"] / float(2))):
6         pyro.sample("Y_chan_totals", dist.Normal(Y_chan_hat, sig_chan), obs=Ychan)
7 # ... similarly for all other parent groups (GT, AREA, CAT, etc.)

```

5.5.5 Inference via SVI

We fit the model with Stochastic Variational Inference (SVI) using the Trace_ELBO objective and a clipped AdamW optimizer with an exponential learning rate scheduler. The guide (variational distribution) uses mean-field Normal distributions for weights and Log-Normal distributions for positive scales, with parameters learned via `pyro.param`.

```

1 # Pyro: Illustrative guide pattern for a parameter
2 Wf_loc = pyro.param("Wp_f_loc", torch.zeros_like(Wp_f_prior_shape))
3 Wf_su  = pyro.param("Wp_f_su", torch.full_like(Wp_f_prior_shape, -1.5))
4 Wf_sc  = torch.nn.functional.softplus(Wf_su) + 1e-5
5 pyro.sample("Wp_f", dist.Normal(Wf_loc, Wf_sc).to_event(2))

```

Point forecasts are generated from the posterior means of the guide's parameters.

5.5.6 Summary of Design Choices

- Factorized Architecture: Modeling factors (Products, Stores, Areas) separately and then composing them is the core design principle for achieving scalability and capturing interaction effects.
- Low-Rank Composition: Replaces a dense head, acting as a powerful structural prior that dramatically reduces parameters from $O(C^2)$ to $O((P + S + A) \cdot r)$.
- Multi-Stage Priors: Layered Bayesian regularization on patch embeddings (temporal decay) and mixers (Minnesota-style shrinkage) stabilizes the feature learning process.
- Covariate Modulation: FiLM and bias heads provide a flexible, low-overhead mechanism for injecting static entity information directly into the dynamics.
- Weighted Hierarchical Likelihood: Supervising latent leaves via weighted parent aggregates allows the model to prioritize key time series and facilitates inductive transfer from stable, high-level signals down to the granular forecasts.

Chapter 6

Simulation Study

In order to evaluate the merits of the proposed framework, we begin with a controlled simulation study. This allows us to isolate and test whether the model correctly captures the intended structural dynamics such as seasonal patterns, product introductions with cannibalization, cross-channel interactions, and heterogeneity across regions and store sizes.

6.1 The Data Generating Process

To rigorously assess the model’s performance, we specified a data generating process (DGP) designed to produce a synthetic panel dataset with known structural properties. The simulation spans five years of daily observations. The data are organized according to a hierarchical structure common in retail contexts: sales are recorded for six unique products across two channels (Retail and Ecommerce), nested within twelve physical stores (for the Retail channel), four distinct geographic areas, and two superordinate regions. This multi-level structure, partially illustrated in Figure 6.1, is intended to induce realistic correlation patterns and test the model’s capacity to handle nested dependencies.

6.1.1 Temporal Dynamics and Stochastic Components

The temporal evolution of the sales series for each product in each location is modeled as a superposition of deterministic and stochastic components. The primary deterministic drivers include:

- **Weekly Periodicity:** Channel-specific, fixed day-of-week effects are included to capture distinct intra-week shopping patterns. The Retail channel exhibits elevated weekend activity, while the Ecommerce channel displays a more uniform pattern.

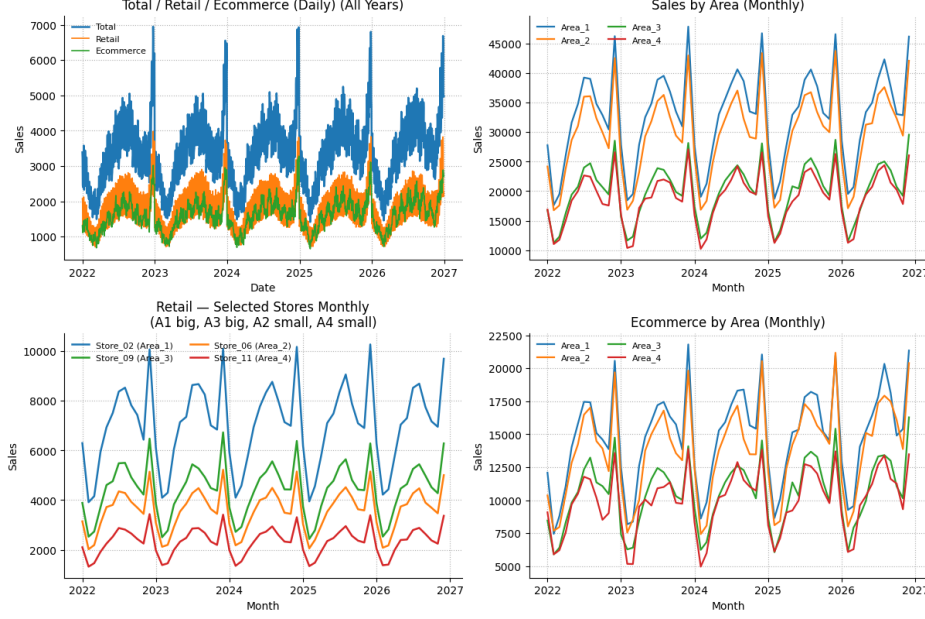


Figure 6.1: Hierarchical Sales Data

- **Annual Seasonality:** A complex, multi-modal annual seasonal curve, common to both channels, was constructed from a sum of Gaussian basis functions. This deterministic function, $f_{yearly}(t)$, is designed to represent complex seasonal demand patterns such as holiday peaks and seasonal lulls.

To introduce more complex dynamics, particularly within the Ecommerce channel, we incorporated an area-specific deterministic linear trend, $d_i \cdot t$, and an autoregressive process of order one, $AR(1)$. The latter component, $z_t = \phi z_{t-1} + \epsilon_t$ with $\phi = 0.65$, was added to the mean sales level to induce serial correlation and mimic the persistence of demand shocks often observed in time series data.

6.1.2 Structural Heterogeneity and Product-Level Dynamics

To reflect the heterogeneity inherent in real-world markets, we specified structural differences across both spatial and product dimensions. The base sales potential was systematically varied across geographic areas and store sizes via multiplicative scaling factors. This ensures that the model is tested against varying scales of magnitude, as shown in the area and store breakouts in Figure 6.1.

The product dimension was engineered to embed specific, challenging dynamics, depicted in Figure 6.2, for the model to identify:

- **Static Channel Affinity (P1 & P2):** These products were assigned inverse base shares

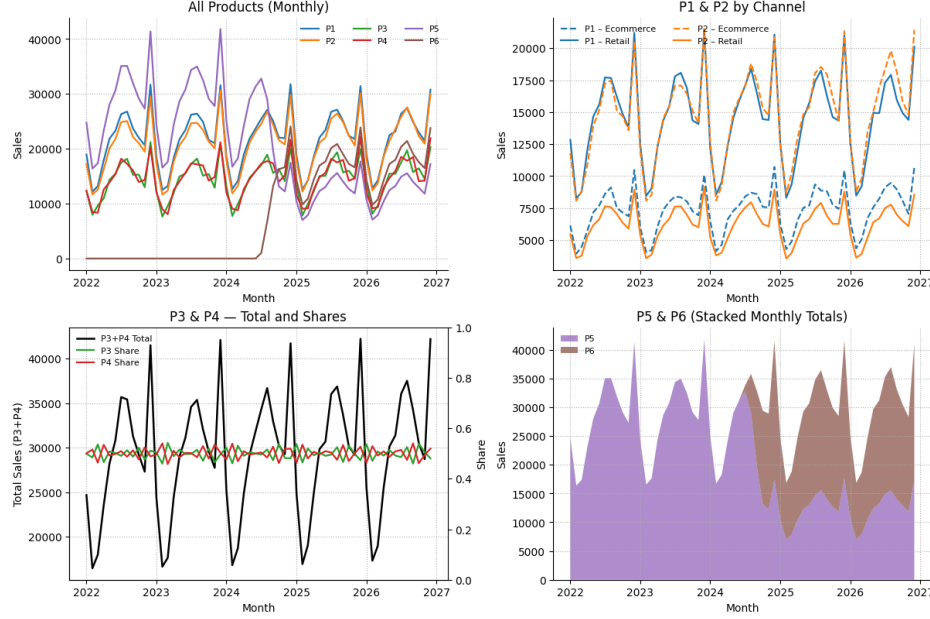


Figure 6.2: Product Hierarchies

in the product mix for each channel. This static differentiation tests the model's ability to estimate persistent cross-channel interaction effects and attribute sales correctly in the presence of strong consumer channel preferences.

- **Competitive Substitution (P3 & P4)**: These products were modeled as direct substitutes. Their combined share of the product mix, W_{34} , was held constant, but this total was allocated between them using a deterministic sinusoidal function of time: $s_3(t) = s_{34}(t) \cdot W_{34}$ and $s_4(t) = (1 - s_{34}(t)) \cdot W_{34}$, where $s_{34}(t) = 0.5 + A \sin(2\pi t/T + \phi)$. This structure mimics phenomena such as alternating promotional schedules and tests for the ability to capture time-varying competitive interactions.
- **Product Cannibalization (P5 & P6)**: This dynamic simulates a new product introduction. At a fixed time t_{intro} , product P6 is launched. Its market share is not incremental but is drawn directly from the incumbent product, P5. The cannibalization rate, $r(t)$, follows a smooth sigmoidal trajectory (a cosine ramp function) over a 120-day launch period, such that the new shares become $s_6(t) = r(t) \cdot s_5^{base}$ and $s_5(t) = (1 - r(t)) \cdot s_5^{base}$. This provides a clear ground truth for evaluating the model's ability to capture the effects of a product launch on existing items.

6.1.3 Observational Noise Model

The final step in the DGP was the introduction of observational noise. The sales value for each series at each time step, y_t , is a random draw from a Gaussian distribution centered at the deterministic mean level, μ_t , described by the components above. A critical feature of this noise model is heteroscedasticity. The standard deviation of the Gaussian noise was specified as a fixed fraction of the conditional mean, such that $\sigma_t = k \cdot \mu_t$ (with $k = 0.18$). This variance structure, where uncertainty scales with the mean, is a common empirical property of sales data. The resulting synthetic dataset thus provides a rich and challenging environment with known ground truth for a comprehensive assessment of the model’s parameter recovery and predictive accuracy.

6.2 Evaluation Protocol

We evaluate models on a five-year (1825 days) simulated retail panel using a fixed temporal split. The first four years (days 1–1460) constitute the training set; the final year (days 1461–1825) is the test set. From the last training timestamp, each model produces a single multi-step forecast for a one-year horizon, $H = 365$ days. Accuracy is computed by comparing these 365 forecasted values to the first 365 days of the test period.

The panel contains 96 leaf series (the most granular product $\times$ location units). All higher-level series (e.g., products, categories, channels, stores, areas, grand total) are deterministic sums of these leaves. For evaluation we align truth and forecast on the same (date, leaf) keys and discard any pairs that are missing on either side.

6.2.1 Metrics and Aggregation Order

Our primary accuracy measure is the Mean Absolute Percentage Error (MAPE). For a univariate series with actuals $\{A_t\}_{t=1}^n$ and forecasts $\{F_t\}_{t=1}^n$,

$$\text{MAPE} = \frac{100}{n} \sum_{t=1}^n \left| \frac{A_t - F_t}{A_t} \right|.$$

We also report the volume-weighted counterpart,

$$\text{WMAPE} = 100 \frac{\sum_{t=1}^n |F_t - A_t|}{\sum_{t=1}^n |A_t|}.$$

In our controlled simulation $A_t > 0$, so percentage errors are well-defined.¹

A key design choice is the order of operations. To assess how accuracy improves with aggregation, we always aggregate first, then compute MAPE. Concretely:

(i) Individual Total (bottom level). Treat each (date, leaf) pair as an observation and compute a single MAPE across all such points:

$$\text{MAPE}_{\text{Ind}} = \frac{100}{|\mathcal{S}| H} \sum_{s \in \mathcal{S}} \sum_{t=1}^H \left| \frac{A_{s,t} - F_{s,t}}{A_{s,t}} \right|,$$

where $\mathcal{S}$ indexes the 96 leaves.

(ii) Aggregate Total (grand total). First sum all leaves by date to form a single aggregate series $A_t^{\text{GT}} = \sum_s A_{s,t}$ and $F_t^{\text{GT}} = \sum_s F_{s,t}$, then compute MAPE/WMAPE on that one series:

$$\text{MAPE}_{\text{GT}} = \frac{100}{H} \sum_{t=1}^H \left| \frac{A_t^{\text{GT}} - F_t^{\text{GT}}}{A_t^{\text{GT}}} \right|, \quad \text{WMAPE}_{\text{GT}} = 100 \frac{\sum_{t=1}^H |F_t^{\text{GT}} - A_t^{\text{GT}}|}{\sum_{t=1}^H |A_t^{\text{GT}}|}.$$

(iii) Level-wise aggregates (e.g., Category, Channel, Area, Region, Store, Product). For a given level L with entities $i \in L$ (e.g., categories $i \in \{A, B, C\}$), we first aggregate leaves within each entity to obtain series

$$A_t^{(i)} = \sum_{s \in i} A_{s,t}, \quad F_t^{(i)} = \sum_{s \in i} F_{s,t}.$$

We then report:

- Per-entity MAPE for each aggregated entity i :

$$\text{MAPE}^{(i)} = \frac{100}{H} \sum_{t=1}^H \left| \frac{A_t^{(i)} - F_t^{(i)}}{A_t^{(i)}} \right|.$$

- Macro-MAPE at level L (equal-weight average across entities):

$$\text{MAPE}_L^{\text{macro}} = \frac{1}{|L|} \sum_{i \in L} \text{MAPE}^{(i)}.$$

¹For real data with zeros or near-zeros one may either impose a minimum denominator or switch to MASE.

- Pooled WMAPE at level L (micro; all entity–days pooled):

$$\text{WMAPE}_L = 100 \frac{\sum_{i \in L} \sum_{t=1}^H |F_t^{(i)} - A_t^{(i)}|}{\sum_{i \in L} \sum_{t=1}^H |A_t^{(i)}|}.$$

This protocol answers the intended question: do errors contract when we move from leaves to coarser aggregates such as categories or channels? Because we aggregate first and only then compute percentage errors, level–wise metrics are not conflated with the grand total.

6.2.2 Implementation Notes

All metrics are computed on the $H=365$ –day horizon. Truth and forecast are aligned on common (date, leaf) keys; dates missing on either side are dropped. When constructing aggregated series we sum over leaves at each date (e.g., Category A, B, C produce three time series), then compute the per–entity MAPEs and the level’s macro–MAPE/pooled–WMAPE as defined above. For completeness we also report *Individual Total* and *Aggregate Total*. The resulting tables list (i) level–wise macro–MAPEs, (ii) the corresponding pooled WMAPEs, and (iii) per–entity MAPEs for interpretability.

6.3 Benchmark Model 1: VAR

A vector autoregression (VAR) is the canonical econometric baseline for multivariate forecasting with contemporaneous cross–series dependence captured through lagged interactions. It serves as a linear, interpretable counterpoint to our proposed factorized deep model: if cross–series structure is largely linear and short–memory, a well–specified VAR should be competitive at coarse aggregates, while its parameter growth and stationarity assumptions typically degrade bottom–level accuracy in high dimensions.

Let $\mathbf{y}_t \in \mathbb{R}^C$ denote the $C=96$ leaf series (daily sales) on day t . The benchmark fits a $\text{VAR}(p)$ with time–constant exogenous regressors:

$$\mathbf{y}_t = \sum_{\ell=1}^p \mathbf{A}_\ell \mathbf{y}_{t-\ell} + \mathbf{B} \mathbf{x} + \boldsymbol{\varepsilon}_t, \quad \boldsymbol{\varepsilon}_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}), \quad (6.1)$$

where $\mathbf{x}$ stacks one–hot category fixed effects (with one column dropped), repeated across t ; trend is omitted (`trend=n`). Model order p is selected by AIC with a small cap (for numerical stability) on the training window.

Retail panels exhibit asynchronous product/channel “births.” To avoid spurious zeros and unbalanced pre-histories, we:

1. identify each series’ first live date (based on an `exist_flag`),
2. restrict the training window to dates $\geq$ the latest first-live date among series that are live at least once during Train (a global birth date), and
3. align the forecast horizon exactly to the `Test` calendar; forecasts for pre-intro days are zeroed by construction.

This yields a well-posed wide panel (date $\times$ series) with daily frequency and mitigates the small-sample instability of large VARs.

Both the VAR and our Hierarchical Bayesian PatchTST allow information to flow across series. The VAR does so densely and linearly through lag matrices $\{\mathbf{A}_\ell\}$, with parameters scaling roughly as $O(C^2p)$; our model replaces dense mixing with a low-rank, factorized head and Bayesian shrinkage, achieving selective (evidence-driven) pooling and hierarchical coherence without quadratic growth. Empirically, the VAR provides a strong sanity check for whether short-lag linear dependencies alone can explain aggregate improvements.

6.4 Benchmark Model 2: Vanilla PatchTST

PatchTST is a state-of-the-art univariate Transformer for long-horizon forecasting. As a benchmark, a vanilla PatchTST that treats each leaf independently isolates the contribution of modern sequence encoders without cross-series mixing or hierarchy-aware training. This allows us to attribute gains in our model specifically to (i) selective cross-series pooling, (ii) soft reconciliation, and (iii) Bayesian regularization.

For each leaf series $s \in \{1, \dots, 96\}$, we form overlapping patches of length P and stride S , project patches to d -dimensional tokens, add sinusoidal positions, and pass through a Transformer encoder with shared weights. A per-series static vector $\mathbf{c}_s$ (one-hot encodings of channel, region, area, store, store size, category, product) is linearly mapped and added to the pooled token:

$$\mathbf{z}_s = \text{Pool}(\text{Encoder}(\text{Proj}(\text{Patchify}(y_{s,1:T})))) + \mathbf{W}_{\text{stat}}\mathbf{c}_s, \quad (6.2)$$

$$\hat{\mathbf{y}}_{s,1:H} = \mathbf{W}_{\text{head}} \text{LN}(\mathbf{z}_s), \quad (6.3)$$

yielding a direct $H=365$ -step forecast per series. Training uses per-series MSE on the Train window via sliding input-target pairs; normalization uses Train statistics. There is no cross-series interaction, no reconciliation loss, and no Bayesian priors.

Vanilla PatchTST shares the same patch-based encoder family but differs along three axes central to our contribution:

1. Cross-series structure: Vanilla is channel-independent; our model adds a low-rank composition across factor groups (products, stores, areas) to capture interactions efficiently.
2. Hierarchy awareness: Vanilla optimizes per-series losses; our model supervises latent leaves through a weighted hierarchical likelihood (soft reconciliation), directly targeting aggregation-level accuracy.
3. Regularization: Vanilla relies on implicit regularization (dropout/weight decay); our model places structured priors (temporal-decay patch priors, Minnesota-style mixers) to implement principled partial pooling.

Comparing against this baseline quantifies the incremental value of selective cross-series pooling and hierarchy-aware training beyond a strong univariate Transformer encoder.

Both benchmarks generate forecasts on the exact **Test** calendar and zero out pre-intro predictions using the same `exist_flag` logic as our model, ensuring apples-to-apples evaluation under the protocol in §6.2.

6.5 Results, Comparison, and Discussion

In this section, we evaluate four forecasting approaches. The baselines are a panel VAR with entity-level exogenous covariates and a Vanilla PatchTST augmented with static entity covariates. We compare these to two variants of the proposed HB-PatchTST: HB-PatchTST-1, which applies equal weighting across hierarchy levels during training, and HB-PatchTST-2, which omits this constraint and allows the model to allocate capacity endogenously.

6.5.1 Metric Table Comparison

Metric	Panel VAR		Vanilla PatchTST		HB-PatchTST-1		HB-PatchTST-2	
	MAPE	WMAPE	MAPE	WMAPE	MAPE	WMAPE	MAPE	WMAPE
Individual Total	70.5%	37.6%	41.4%	33.8%	81.2%	50.4%	81.1%	49.0%
Aggregate Total	25.3%	21.8%	6.7%	6.6%	6.4%	6.5%	30.4%	29.5%
Category A	25.3%	21.7%	5.0%	4.9%	8.0%	8.0%	33.0%	34.5%
Category B	25.2%	21.7%	5.6%	5.9%	6.9%	6.9%	29.4%	27.1%
Category C	26.2%	22.4%	20.8%	20.0%	6.6%	6.5%	31.5%	28.3%
Aggregate Category	25.6%	21.9%	10.5%	9.5%	7.2%	7.3%	31.3%	30.6%
Product 1	26.7%	23.0%	5.3%	5.2%	9.4%	9.0%	32.9%	34.8%
Product 2	25.4%	21.8%	7.1%	7.0%	9.4%	9.6%	33.5%	34.6%
Product 3	26.5%	22.7%	7.1%	7.3%	10.8%	10.9%	30.3%	27.8%
Product 4	27.9%	24.7%	7.0%	7.2%	11.1%	11.1%	30.7%	28.8%
Product 5	25.6%	21.4%	95.7%	94.8%	80.4%	72.0%	37.8%	30.5%
Product 6	27.4%	23.9%	56.0%	60.2%	59.7%	54.9%	31.7%	31.4%
Aggregate Product	26.6%	22.9%	29.7%	26.0%	30.1%	24.8%	32.8%	31.8%
Ecommerce	24.6%	21.3%	7.7%	7.7%	10.6%	10.8%	35.4%	35.6%
Retail	30.6%	26.3%	8.0%	7.6%	4.4%	4.3%	29.2%	27.2%
Aggregate Channel	27.6%	23.8%	7.8%	7.6%	7.5%	7.6%	32.3%	31.3%

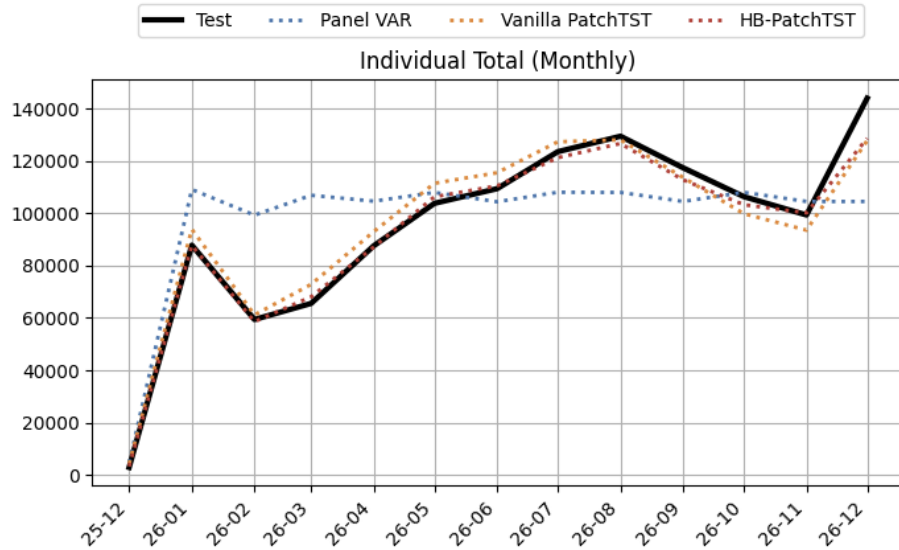


Figure 6.3: Totals

Overall patterns in the metric table indicate that the panel VAR, absent explicit seasonal regressors, systematically underfits periodic structure and consequently overgeneralizes, particularly at the bottom level. By contrast, the Vanilla PatchTST achieves a strong fit on this synthetic panel, albeit in a configuration that does not scale favorably beyond small settings; it excels on several product-level series (e.g., Products 3 and 4) but fails to capture cannibalization dynamics in Products 5 and 6.

The hierarchy-aware variants sharpen these trade-offs. HB-PatchTST-1 delivers the strongest aggregate performance, markedly improving WMAPE at the total and category aggregates relative to both baselines. This suggests that equal weighting across hierarchical levels acts as an effective regularizer, encouraging coherent forecasts that respect aggregation structure while retaining flexibility at lower levels.

HB-PatchTST-2, which removes equal hierarchy weighting, exhibits weaker seasonal fit yet demonstrates superior generalization to the cannibalization regime of Products 5 and 6. This pattern is consistent with a capacity reallocation toward cross-series competitive effects at the expense of fine-grained seasonal reconstruction, indicating a controllable bias–variance trade-off induced by hierarchy weighting.

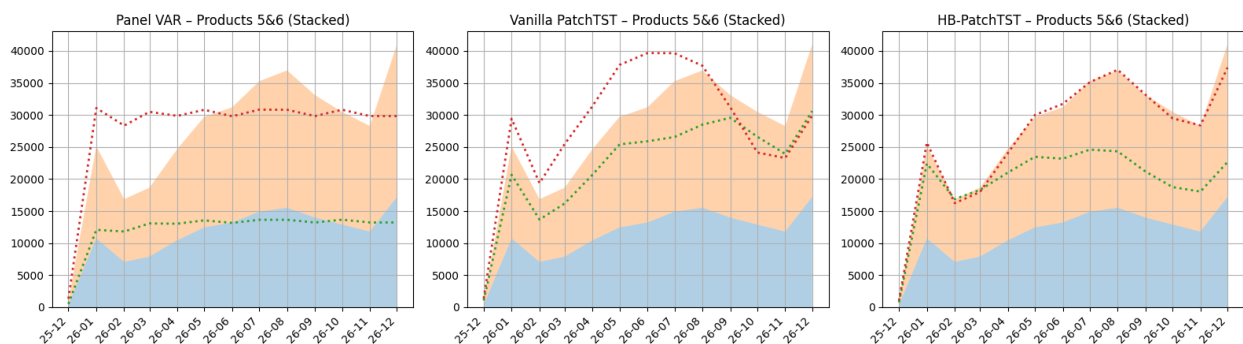


Figure 6.4: Products 5 & 6

Figure 6.4 illustrates the differential behavior on Category C: the Vanilla model largely misses the antagonistic interaction between Products 5 and 6, whereas HB-PatchTST captures the direction and magnitude of cannibalization more faithfully, especially under the unweighted variant.

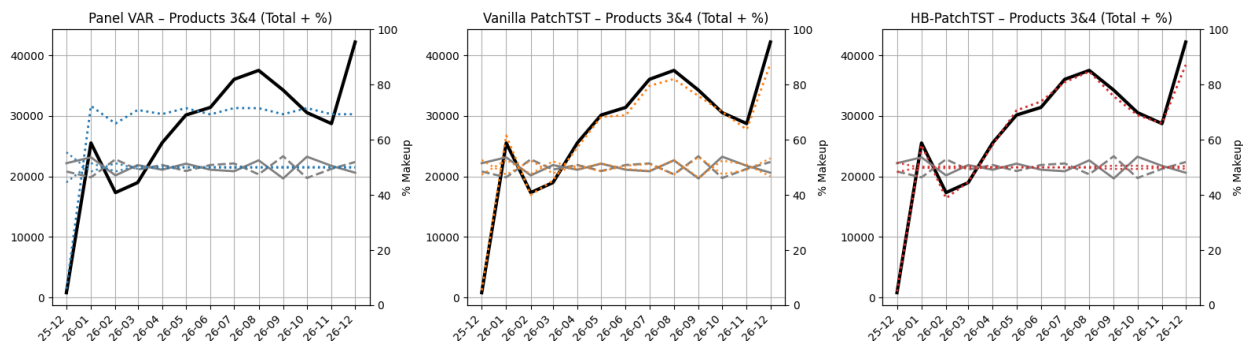


Figure 6.5: Products 3 & 4

Conversely, Figure 6.5 shows a setting (Category B) where Vanilla PatchTST matches the relative offsets between Products 3 and 4 with remarkable fidelity; here HB-PatchTST

improves aggregate coherence but does not fully reproduce the inter-series phase and level differences at the bottom level.

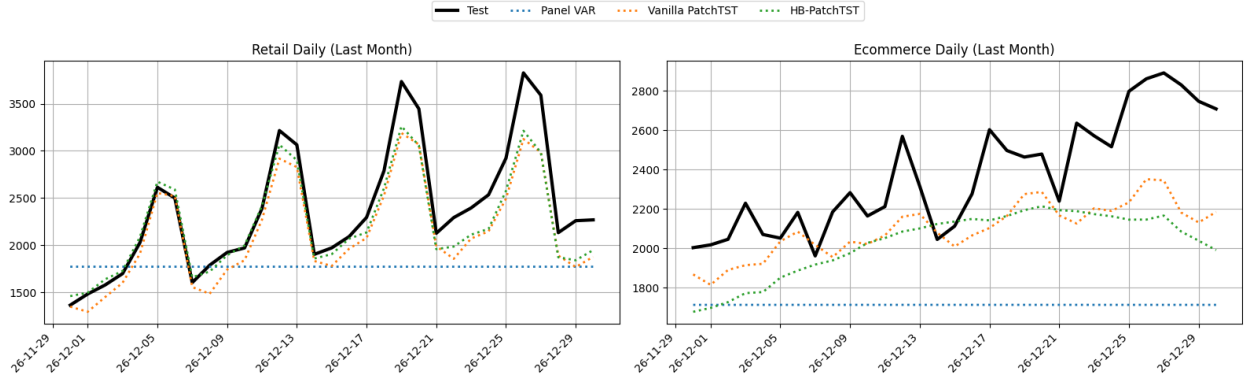


Figure 6.6: Weekly Seasonality Excerpt

Finally, the weekly excerpt in Figure 6.6 highlights that both Vanilla PatchTST and HB-PatchTST attain broadly comparable within-week fit, though neither adequately reproduces the volatility profile in the Ecommerce channel. In particular, HB-PatchTST tends to underfit this high-frequency component when hierarchy weighting is enforced, reflecting its emphasis on aggregate consistency.

Taken together, these results underscore the value of embedding hierarchical structure directly into a transformer-based forecaster: equal hierarchy weighting improves coherence and aggregate accuracy, while relaxing this constraint can enhance sensitivity to cross-series interactions such as cannibalization. From an econometric perspective, the hierarchy weighting plays a role akin to structured regularization or partial pooling, and in combination with modern sequence models, offers a principled mechanism to balance fit at granular levels with stability under aggregation.

A final caveat concerns model capacity and potential overfitting. The apparent leaf-level advantage of the Vanilla PatchTST may, in part, reflect its high effective capacity relative to per-series sample size, enabling it to memorize idiosyncratic harmonics present in the synthetic panel rather than learning stable cross-series structure. By contrast, HB-PatchTST attains competitive accuracy without directly training on all $L = 96$ bottom-level series, instead leveraging hierarchical parameter sharing and weighting to induce a parsimonious inductive bias. This ability to approach the Vanilla PatchTST’s performance while forgoing explicit leaf-wise fitting is consequential for real-world applications with $L \gg 10^3$ leaves, where memory and sample-complexity constraints make fully expressive per-series models impractical. In such regimes, HB-PatchTST’s factorization offers a scalable pathway to coherent forecasts that balance robustness and granularity.

6.5.2 Other Tables and Plots

Metric	Panel VAR		Vanilla PatchTST		HB-PatchTST-1		HB-PatchTST-2	
	MAPE	WMAPE	MAPE	WMAPE	MAPE	WMAPE	MAPE	WMAPE
Area_1	26.3%	22.2%	8.4%	8.1%	8.4%	8.1%	31.8%	32.3%
Area_2	26.6%	22.7%	8.6%	8.4%	7.6%	7.6%	32.3%	31.7%
Area_3	25.7%	22.4%	9.0%	8.8%	9.0%	9.0%	29.4%	27.6%
Area_4	26.6%	22.5%	8.5%	8.3%	9.4%	9.2%	31.1%	27.5%
Aggregate Area	26.3%	22.5%	8.6%	8.4%	8.6%	8.3%	31.1%	30.3%
Region A	25.7%	21.9%	7.4%	7.2%	6.6%	6.5%	31.6%	31.7%
Region B	25.5%	22.0%	7.3%	7.2%	7.8%	7.8%	29.7%	27.2%
Aggregate Region	25.6%	21.9%	7.3%	7.2%	7.2%	7.0%	30.6%	29.9%
Online 1	27.6%	22.8%	12.0%	11.5%	15.1%	14.7%	38.4%	40.2%
Online 2	28.4%	24.1%	12.5%	12.1%	14.2%	14.2%	37.9%	39.4%
Online 3	27.1%	23.4%	13.4%	12.9%	14.9%	14.5%	36.4%	34.1%
Online 4	29.0%	24.1%	13.0%	12.6%	16.4%	15.7%	38.1%	33.1%
Store 1	32.5%	27.7%	11.0%	10.3%	8.2%	8.0%	30.8%	31.1%
Store 2	31.4%	26.9%	10.3%	9.8%	8.2%	8.0%	30.3%	31.2%
Store 3	32.1%	27.4%	10.9%	10.3%	7.8%	7.7%	40.5%	32.5%
Store 4	31.6%	27.0%	10.0%	9.6%	7.5%	7.3%	32.8%	30.6%
Store 5	31.2%	26.5%	10.1%	9.8%	8.1%	7.9%	29.2%	28.6%
Store 6	32.3%	27.6%	11.3%	10.4%	8.0%	8.0%	42.8%	34.0%
Store 7	30.8%	26.6%	9.4%	9.2%	7.4%	7.4%	29.7%	26.0%
Store 8	31.6%	26.9%	10.9%	10.4%	7.6%	7.7%	30.1%	26.2%
Store 9	30.8%	26.3%	9.8%	9.4%	7.7%	7.5%	28.9%	27.2%
Store 10	31.5%	27.1%	10.4%	10.1%	7.6%	7.8%	31.8%	27.3%
Store 11	31.6%	27.1%	11.0%	10.5%	8.1%	8.0%	30.4%	26.4%
Store 12	30.9%	26.6%	9.7%	9.5%	7.1%	7.1%	29.1%	27.2%
Aggregate Store	30.7%	25.3%	11.0%	11.0%	9.6%	11.2%	33.6%	33.4%

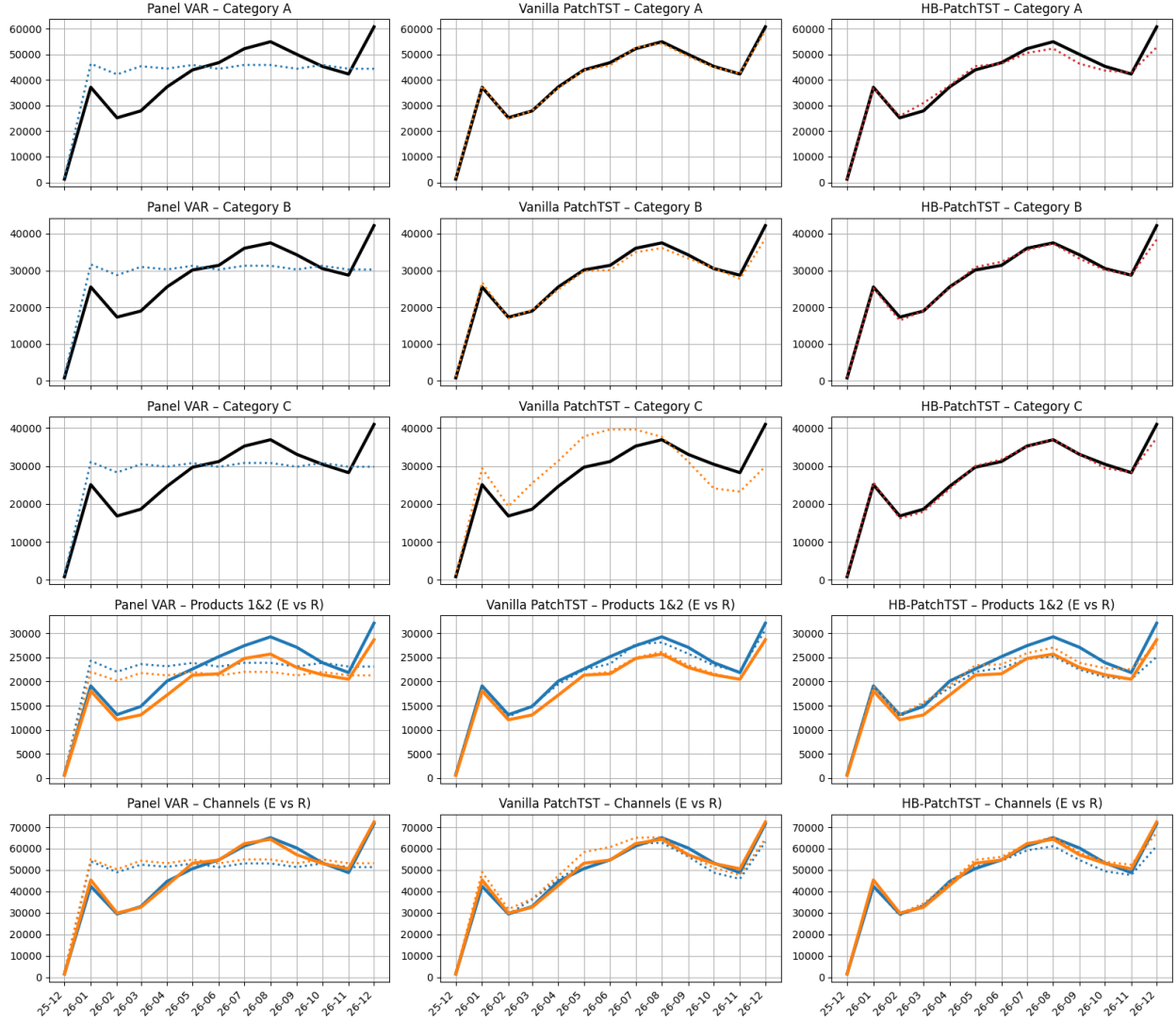


Figure 6.7: Product and Channel

Chapter 7

Discussion and Conclusion

This thesis introduced the Hierarchical Bayesian Patch Time Series Transformer (HB-PatchTST), a novel architecture designed to address the persistent challenge of forecasting high-dimensional, hierarchically structured time series. By synthesizing principles from modern deep learning and classical Bayesian econometrics, the proposed model navigates the trade-off between the statistical efficiency of channel-independent models and the expressive power of fully mixed systems. The empirical evaluation conducted on a controlled simulation study with complex, known dynamics has demonstrated the model’s capacity to generate accurate, coherent, and probabilistically sound forecasts.

7.1 Strengths and Limitations

The primary strength of the HB-PatchTST framework lies in its factorized, low-rank architecture. This design choice successfully circumvents the quadratic parameter growth, $O(C^2)$, that renders traditional multivariate models like VARs and fully mixed Transformers statistically and computationally prohibitive in large panels. By learning disentangled representations for underlying factors (e.g., products, stores, areas) and composing them through an efficient tensor interaction head, the model achieves a principled compromise between expressivity and scalability.

A second significant advantage is the use of a structured Bayesian front-end. The imposition of temporally decaying priors on patch embeddings and Minnesota-style shrinkage on within-group mixers provides robust inductive biases. This regularizes the feature extraction process, encouraging the model to privilege recent information and favor own-series dynamics unless strong cross-series evidence is present. This stands in contrast to the often unstructured regularization (e.g., dropout) employed in standard deep learning models.

Furthermore, the implementation of soft reconciliation via a weighted hierarchical like-

likelihood marks a conceptual advance over conventional post-hoc reconciliation methods. By treating leaf-level series as latent variables supervised by their observed aggregates during training, the model learns to produce forecasts that are inherently coherent across the hierarchy. The weighting mechanism offers a valuable tool for practitioners to prioritize accuracy at strategically important aggregation levels.

However, the model is not without its limitations. First, despite its relative efficiency, the computational overhead of training a deep Bayesian model with Stochastic Variational Inference (SVI) remains substantially higher than that of classical econometric models or non-Bayesian deep learning baselines. Second, the efficacy of the Bayesian front-end is contingent upon the specification of hyperparameters for the structured priors (e.g., temporal decay rate, shrinkage tightness). While these priors provide beneficial structure, their calibration may require domain expertise or further computational investment in empirical Bayes or cross-validation routines. Finally, while the factorized design enhances interpretability relative to a monolithic “black-box” model, the internal workings of the Transformer encoder remain less transparent than the coefficient matrices of a linear VAR.

7.2 Summary of Contributions

This thesis makes four primary contributions to the field of hierarchical time series forecasting:

- **A Novel Factorized Architecture:** We proposed a factorized variant of the PatchTST model with a low-rank composition head. This design captures salient cross-factor interactions without incurring the quadratic complexity of dense channel mixing, thereby enhancing scalability and statistical efficiency.
- **Principled Bayesian Regularization:** We introduced structured priors at the model’s front-end, including temporal-decay patch embeddings and Minnesota-style mixers. This provides a robust mechanism for partial pooling that defaults to parsimonious own-series behavior while permitting data-driven statistical borrowing across related series.
- **Hierarchy-Aware Training Objective:** We formulated a weighted hierarchical likelihood that supervises latent leaf-level forecasts through their observed aggregates. This “soft reconciliation” approach embeds hierarchical coherence directly into the training process, improving multi-level consistency.

- **Empirical Validation and Scalable Uncertainty:** Through a simulation study with known structural dynamics, we demonstrated that the proposed model can effectively capture complex interactions such as product cannibalization. The use of SVI delivers scalable posterior predictive distributions, enabling risk-aware decision-making in high-dimensional panels.

7.3 Implications for Practice and Theory

For practitioners in fields such as retail, supply chain management, and finance, this work offers a scalable and powerful framework for generating coherent forecasts in complex, high-dimensional systems. The ability to produce probabilistic forecasts at all levels of a hierarchy supports more sophisticated inventory management, resource allocation, and risk assessment. The model’s design, which explicitly separates temporal dynamics from static attributes, provides a clear path for injecting domain-specific knowledge.

Theoretically, this research contributes to the growing body of literature at the intersection of deep learning and probabilistic modeling. It demonstrates that the principles of structured shrinkage and hierarchical modeling, long central to Bayesian econometrics, can be effectively transposed to regularize and guide state-of-the-art sequence models like the Transformer. The concept of using a hierarchical likelihood as a form of latent variable supervision offers a compelling alternative to post-hoc reconciliation, suggesting a new paradigm for building inherently coherent forecasting systems.

7.4 Recommendations for Future Directions

Building upon the foundation established in this thesis, several avenues for future research appear promising. First, the model could be extended to incorporate time-varying exogenous covariates (e.g., prices, promotions) by integrating mechanisms such as the gating layers found in the Temporal Fusion Transformer. Second, exploring more adaptive and flexible shrinkage priors, such as global-local horseshoe priors, could allow the model to more effectively handle sparse interaction structures. Third, while SVI provides a scalable inference method, future work could investigate the application of alternative approximate inference techniques, such as normalizing flows, to capture more complex posterior geometries. Finally, the validation of this framework on large-scale, real-world benchmark datasets (e.g., the M5 forecasting competition data) is a critical next step to confirm its practical utility and robustness beyond the controlled simulation environment.

Bibliography

- [1] H. Lütkepohl, *New Introduction to Multiple Time Series Analysis*. Springer, 2005. [Online]. Available: <https://link.springer.com/book/10.1007/978-3-540-27752-1>
- [2] R. B. Litterman, “Forecasting with bayesian vector autoregressions: Five years of experience,” *Journal of Business & Economic Statistics*, vol. 4, no. 1, pp. 25–38, 1986.
- [3] M. Bańbura, D. Giannone, and L. Reichlin, “Large bayesian vector autoregressions,” *Journal of Applied Econometrics*, vol. 25, no. 1, pp. 71–92, 2010.
- [4] L. M. Berliner, “Hierarchical Bayesian time series models,” in *Maximum Entropy and Bayesian Methods*, ser. Fundamental Theories of Physics, K. M. Hanson and R. N. Silver, Eds. Dordrecht: Springer, 1996, vol. 79, pp. 15–22.
- [5] M. Bedard, M. Johnson, and P. Sangrey, “Cross-sectional state-space forecasting with partial pooling,” Amazon Science (Conference: NABE 2022), 2022, online; accessed 2025-08-29. [Online]. Available: <https://www.amazon.science/publications/cross-sectional-state-space-forecasting-with-partial-pooling>
- [6] B. Lim, S. Ö. Arik, N. Loeff, and T. Pfister, “Temporal fusion transformers for interpretable multi-horizon time series forecasting,” *International Journal of Forecasting*, vol. 37, no. 4, pp. 1748–1764, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0169207021000637>
- [7] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, “A time series is worth 64 words: Long-term forecasting with transformers,” in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: <https://openreview.net/forum?id=Jbdc0vTOcol>
- [8] Y. Zhang and J. Yan, “Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting,” in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: <https://openreview.net/forum?id=vSVLM2j9eie>

- [9] R. J. Hyndman, R. A. Ahmed, G. Athanasopoulos, and H. L. Shang, “Optimal combination forecasts for hierarchical time series,” *Computational Statistics & Data Analysis*, vol. 55, no. 9, pp. 2579–2589, 2011.
- [10] S. L. Wickramasuriya, G. Athanasopoulos, and R. J. Hyndman, “Optimal forecast reconciliation for hierarchical and grouped time series through trace minimization,” *Journal of the American Statistical Association*, vol. 114, no. 526, pp. 804–819, 2019.
- [11] C. Challu, K. G. Olivares, B. N. Oreshkin, F. G. Ramírez, M. M. Canseco, and A. Dubrawski, “N-hits: Neural hierarchical interpolation for time series forecasting,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 6, 2023, pp. 6989–6997. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/25854>
- [12] S. Wang, F. Zhou, Y. Sun, L. Ma, J. Zhang, Y. Zheng, B. Zheng, L. Lei, and Y. Hu, “End-to-end modeling of hierarchical time series using autoregressive transformer and conditional normalizing flow-based reconciliation,” *arXiv*, 2022. [Online]. Available: <https://arxiv.org/abs/2212.13706>
- [13] M. Chen, L. Shen, H. Fu, Z. Li, J. Sun, and C. Liu, “Generative pretrained hierarchical transformer for time series forecasting,” in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2024.
- [14] S. Zhao, M. Jin, Z. Hou, C. Yang, Z. Li, Q. Wen, and Y. Wang, “Himtm: Hierarchical multi-scale masked time series modeling with self-distillation for long-term forecasting,” in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM)*, 2024, pp. 3352–3362.
- [15] M. Wang, W. Chen, and B. Chen, “Considering nonstationary within multivariate time series with variational hierarchical transformer for forecasting,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 14, 2024, pp. 15 563–15 570.
- [16] R. Azorin, Z. B. Houidi, M. Gallo, A. Finamore, and P. Michiardi, “Fine-grained attention in hierarchical transformers for tabular time-series,” *arXiv preprint arXiv:2406.15327*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.15327>
- [17] J. L. Elman, “Finding structure in time,” *Cognitive science*, vol. 14, no. 2, pp. 179–211, 1990.
- [18] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

- [19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Łukasz Kaiser, and I. Polosukhin, “Attention is all you need,” *arXiv preprint arXiv:1706.03762*, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>

Appendix A

Data Generating Process

```
1 def make_synthetic_panel(seed: int = 42, n_years: int = 5) -> pd.DataFrame:
2     """
3     Generate synthetic hierarchical retail/ecommerce daily sales panel.
4
5     Returns:
6         pd.DataFrame with columns (in order):
7         ['date', 'series', 'exist_flag', 'channel', 'region', 'area', 'store', 'store_size', 'category', 'product', 'sales']
8     """
9     rng = np.random.default_rng(seed)
10
11     # ----- config -----
12     START = datetime(2022, 1, 1)
13     N_DAYS = n_years * 365
14
15     AREAS = [f"Area_{i+1}" for i in range(4)]
16     STORES = [f"Store_{i+1:02d}" for i in range(12)]
17     AREA_FOR_STORE = {STORES[i]: AREAS[i//3] for i in range(12)}
18
19     # weekday effects (Mon..Sun)
20     WEEKLY_RETAIL = np.array([0.78, 0.82, 0.86, 0.90, 1.08, 1.35, 1.28])
21     WEEKLY_ECOM = np.array([0.97, 0.99, 1.00, 1.00, 1.03, 1.05, 1.05])
22
23     # base scales
24     RETAIL_BASE, ECOM_BASE = 120.0, 135.0
25
26     # area/channel scale
27     AREA_SCALE_RETAIL = {"Area_1": 1.40, "Area_2": 1.28, "Area_3": 0.90, "Area_4": 0.85}
28     AREA_SCALE_ECOM = {"Area_1": 1.35, "Area_2": 1.25, "Area_3": 0.95, "Area_4": 0.90}
29
```

```

30     # store-size factors per area: A1/A2 = 2 big + 1 small; A3/A4 = 2 small + 1 big
31     BIG_F, SMALL_F = 1.35, 0.75
32
33     # product mix shares (pooled shares before dynamics; normalized daily after
34     ↪ adjustments)
35     BASE_MIX_RETAIL = {"P1":0.28, "P2":0.12, "P34":0.28, "P5":0.32}
36     BASE_MIX_ECOM    = {"P1":0.15, "P2":0.30, "P34":0.30, "P5":0.25}
37
38     # region mapping
39     region_map =
40     ↪ {"Area_1":"Region_A", "Area_2":"Region_A", "Area_3":"Region_B", "Area_4":"Region_B"}
41
42     # intro date for P6 cannibalizing P5
43     intro_start = datetime(2024, 7, 1)
44
45     # --- E-commerce global trend target (~+5% over the full span) ---
46     USE_EXP_DRIFT      = True
47     ECOM_TARGET_GROWTH = 0.15 # ~5% total growth from first to last day
48     ECOM_DRIFT_JITTER  = 0.0  # ±fractional jitter around the daily drift per area (0.0
49     ↪ = none)
50
51     # ----- helpers -----
52     def bump(c, w, a, x):
53         return a * np.exp(-0.5 * ((x - c) / w) ** 2)
54
55     def strong_yearly_factor(n_years_: int) -> np.ndarray:
56         d = np.arange(365)
57         y = (bump(359,12,1.2,d) + bump(196,30,0.35,d) + bump(244,22,0.30,d) -
58             ↪ bump((31+90)/2,28,0.70,d))
59         y = (y - y.min()) / (y.max() - y.min())
60         y = 0.55 + 1.10 * y
61         return np.tile(y, n_years_)
62
63     # reproducible "deterministic uniform" tied to (seed, key)
64     def detu(lo: float, hi: float, key: str) -> float:
65         s = f"{seed}:{key}".encode()
66         h = int.from_bytes(hashlib.blake2b(s, digest_size=8).digest(), "little")
67         local = np.random.default_rng(h)
68         return lo + (hi - lo) * local.random()
69
70     def store_size_factors() -> dict:
71         f = {}
72         for ai, area in enumerate(AREAS, 1):
73             sts = sorted([s for s in STORES if AREA_FOR_STORE[s] == area])
74             sizes = [BIG_F,BIG_F,SMALL_F] if ai in (1,2) else [SMALL_F,SMALL_F,BIG_F]

```

```

71         for s, sc in zip(sts, sizes):
72             f[s] = sc * detu(0.97, 1.03, f"{area}-{s}")
73     return f
74
75     # ----- precompute daily factors -----
76     dates = [START + timedelta(days=i) for i in range(N_DAYS)]
77     dow    = np.array([d.weekday() for d in dates])
78     yfac   = strong_yearly_factor(n_years)
79     store_scale = store_size_factors()
80
81     # --- e-commerce daily drift (same across areas, optional tiny jitter) ---
82     T = max(N_DAYS - 1, 1)
83     daily_drift = np.log1p(ECOM_TARGET_GROWTH) / T
84     area_drift = {
85         a: daily_drift * (1.0 + detu(-ECOM_DRIFT_JITTER, ECOM_DRIFT_JITTER,
86             ↪ f"edrftj-{a}"))
87         for a in AREAS
88     }
89
90     rows = []
91
92     # ----- Retail: per STORE × PRODUCT -----
93     for s in STORES:
94         a = AREA_FOR_STORE[s]
95         base_unit = RETAIL_BASE * AREA_SCALE_RETAIL[a] * store_scale[s]
96         # P3/P4 inverse share driver per-area (fast sinusoid)
97         phase34 = detu(0, 2*np.pi, f"r34-{a}")
98         for i, d in enumerate(dates):
99             level = base_unit * WEEKLY_RETAIL[dow[i]] * yfac[i]
100             pool = dict(BASE_MIX_RETAIL)
101
102             # dynamic split of P34 into (P3,P4)
103             s34 = 0.5 + 0.40 * np.sin(2*np.pi*i/45.0 + phase34)
104             w34 = pool["P34"]; p3_share, p4_share = s34 * w34, (1.0 - s34) * w34
105
106             # P6 cannibalization of P5 (smooth ramp post-intro)
107             if d >= intro_start:
108                 t = (d - intro_start).days
109                 r = 0.5 * (1 - np.cos(np.pi * min(t/120.0, 1.0))) * 0.6 # up to 60%
110                 p6_share = r * pool["P5"]
111                 p5_share = (1.0 - r) * pool["P5"]
112             else:
113                 p5_share, p6_share = pool["P5"], 0.0
114
115     shares = {

```

```

115         "P1": pool["P1"], "P2": pool["P2"],
116         "P3": p3_share,    "P4": p4_share,
117         "P5": p5_share,    "P6": p6_share
118     }
119     norm = sum(shares.values())
120
121     for p, sh in shares.items():
122         mean = level * (sh / norm)
123         sd    = 0.18 * mean # heteroscedastic
124         val   = max(0.0, mean + rng.normal(0, sd))
125         rows.append((d, "Retail", a, s, p, val))
126
127     # ----- Ecommerce: per AREA × PRODUCT -----
128     phi, shock_sd = 0.65, 12.0
129     for a in AREAS:
130         base_unit = ECOM_BASE * AREA_SCALE_ECOM[a]
131         phase34   = detu(0, 2*np.pi, f"e34-{a}")
132         phase_pulse = detu(0, 2*np.pi, f"epulse-{a}")
133         amp_pulse   = detu(0.08, 0.18, f"eamp-{a}")
134         drift_rate  = area_drift[a]
135         ar = 0.0
136
137     for i, d in enumerate(dates):
138         # base seasonality
139         level = base_unit * WEEKLY_ECOM[dow[i]] * yfac[i]
140         level *= (1 + amp_pulse * np.sin(2*np.pi*i/37.0 + phase_pulse))
141
142         # exponential trend (~+5% total over span)
143         trend = np.exp(drift_rate * i) if USE_EXP_DRIFT else (1.0 + drift_rate * i)
144         level *= trend
145
146         # AR noise
147         ar = phi * ar + rng.normal(0, shock_sd)
148         level = max(0.0, level + ar)
149
150         pool = dict(BASE_MIX_ECOM)
151         s34 = 0.5 + 0.45 * np.sin(2*np.pi*i/52.0 + phase34)
152         w34 = pool["P34"]; p3_share, p4_share = s34 * w34, (1.0 - s34) * w34
153
154         if d >= intro_start:
155             t = (d - intro_start).days
156             r = 0.5 * (1 - np.cos(np.pi * min(t/120.0, 1.0))) * 0.55 # up to 55%
157             p6_share = r * pool["P5"]
158             p5_share = (1.0 - r) * pool["P5"]
159         else:

```

```

160         p5_share, p6_share = pool["P5"], 0.0
161
162     shares = {
163         "P1": pool["P1"], "P2": pool["P2"],
164         "P3": p3_share,   "P4": p4_share,
165         "P5": p5_share,   "P6": p6_share
166     }
167     norm = sum(shares.values())
168
169     # Ecommerce store naming: Online_01..Online_04 (matching Area_1..Area_4)
170     online_store = f"Online_{int(a.split('_')[1]):02d}"
171
172     for p, sh in shares.items():
173         mean = level * (sh / norm)
174         sd    = 0.18 * mean
175         val   = max(0.0, mean + rng.normal(0, sd))
176         rows.append((d, "Ecommerce", a, online_store, p, val))
177
178     # ----- assemble -----
179     df = pd.DataFrame(rows, columns=["date", "channel", "area", "store", "product", "sales"])
180
181     # Balance retail vs ecommerce totals (kept as before)
182     df.loc[df["channel"]=="Ecommerce", "sales"] *= 2.5
183
184     # region
185     df["region"] = df["area"].map(region_map)
186
187     # store_size
188     store_size_map = {}
189     for area in AREAS:
190         sts = sorted([st for st in STORES if AREA_FOR_STORE[st]==area])
191         sizes = ["Large", "Large", "Small"] if area in ("Area_1", "Area_2") else
192             ↪ ["Small", "Small", "Large"]
193         for st, sz in zip(sts, sizes):
194             store_size_map[st] = sz
195     df["store_size"] = df["store"].map(store_size_map)
196     df.loc[df["channel"]=="Ecommerce", "store_size"] = "Online"
197
198     # category
199     category_map = {"P1": "Category_A", "P2": "Category_A", "P3": "Category_B",
200                    "P4": "Category_B", "P5": "Category_C", "P6": "Category_C"}
201     df["category"] = df["product"].map(category_map)
202
203     # stable series ids (explicit ecommerce store IDs included)
204     df["series"] = (

```

```

204         "Series_"
205     + df.groupby(["channel","area","store","product"],
206         ↪ sort=False).ngroup().add(1).astype(str)
207
208     )
209
210     # exist_flag: 1 from first day that series has positive sales onward (P6=0 pre-intro)
211     first_live = (df.loc[df["sales"] > 0]
212         .groupby("series")["date"]
213         .min())
214     df["exist_flag"] = (df["date"] >= df["series"].map(first_live)).astype(int)
215     split_date = df["date"].min() + timedelta(days=(n_years-1)*365)
216     df["scenario"] = np.where(df["date"] < split_date, "Train", "Test")
217     # final column order
218     df = df[[
219         "scenario", "date", "series", "exist_flag", "channel", "region", "area", "store",
220         "store_size", "category", "product", "sales"
221     ]]
222
223     return df

```

Appendix B

BH-PatchTST

```
1 def hbptst_forecast(
2     df: pd.DataFrame,
3     lookback: int = 365,
4     horizon_cap: int = 365,
5     patch_len: int = 14,
6     stride: int = 7,
7     d_model: int = 128,
8     nhead: int = 8,
9     nlayers: int = 4,
10    dff: int = 512,
11    dropout: float = 0.10,
12    batch_size: int = 32,
13    epochs: int = 200,
14    lr: float = 3e-4,
15    rank: int = 4,
16    film_scale: float = 0.10,    # = 1 + film_scale * tanh(...)
17    bias_bottleneck: int = 8,    # r for horizon-wise bias head
18    seed: int = 42,
19 ) -> Tuple[pd.DataFrame, pd.DataFrame]:
20     """
21     HB-PatchTST (asymmetric) with BOTH FiLM (feature-wise modulation) and
22     horizon-wise bias injection from static covariates.
23
24     Trains on Train period and appends bottom-level forecasts for Test dates
25     with scenario='hbptst_forecast'.
26
27     Returns:
28         df_out: original df + forecasts (same column order as input)
29         train_log: DataFrame with per-epoch ELBO and quick validation MAPEs
30     """
```

```

31 import numpy as np
32 import pandas as pd
33 import torch
34 import torch.nn as nn
35 from torch.utils.data import Dataset, DataLoader
36 import pyro
37 import pyro.distributions as dist
38 from pyro.infer import SVI, Trace_ELBO
39 from pyro.optim import ClippedAdam
40 from pyro.optim import ExponentialLR
41
42 # ----- checks / setup -----
43 required = ["scenario", "date", "series", "exist_flag", "channel", "region", "area",
44            "store", "store_size", "category", "product", "sales"]
45 miss = [c for c in required if c not in df.columns]
46 if miss:
47     raise ValueError(f"df missing columns: {miss}")
48
49 torch.manual_seed(seed)
50 np.random.seed(seed)
51 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
52
53 dfx = df.copy()
54 dfx["date"] = pd.to_datetime(dfx["date"])
55
56 # canonicalize channel
57 def _canon_channel(x: object) -> str:
58     s = str(x).strip().lower()
59     if s in {"ecommerce", "e-comm", "ecomm", "e-commerce", "online", "web"}:
60         return "ecommerce"
61     if s in {"retail", "store", "in-store", "brick&mortar", "brick and mortar"}:
62         return "retail"
63     return s
64 dfx["channel_c"] = dfx["channel"].map(_canon_channel)
65
66 # Exact Test calendar
67 test_dates =
68     ↪ pd.DatetimeIndex(sorted(dfx.loc[dfx["scenario"]=="Test", "date"].unique()))
69 if len(test_dates) == 0:
70     raise ValueError("No Test dates found in df['scenario']=='Test'.")
71
72 # Canonical product order (6)
73 prod_order = ["P1", "P2", "P3", "P4", "P5", "P6"]
74 if not set(prod_order).issubset(set(dfx["product"].astype(str))):
75     raise ValueError("Expected products P1..P6 in df['product'].")

```

```

75
76 # Split by channel
77 dfr = dfx[dfx["channel_c"].eq("retail")].copy()
78 dfe = dfx[dfx["channel_c"].eq("ecommerce")].copy()
79 if dfr.empty or dfe.empty:
80     raise ValueError(
81         "Need both Retail and Ecommerce data in df['channel'] "
82         f"(normalized counts:
83         ↪ {dfx['channel_c'].value_counts(dropna=False).to_dict()})"
84     )
85
86 # Sets
87 stores_r = sorted([s for s in dfr["store"].fillna("").astype(str).unique() if
88     ↪ s.strip()!=""])
89 areas_e = sorted([a for a in dfe["area"].fillna("").astype(str).unique() if
90     ↪ a.strip()!=""])
91 if len(stores_r)==0 or len(areas_e)==0:
92     raise ValueError("Need at least 1 retail store and 1 ecommerce area in df.")
93
94 P = len(prod_order)
95 S = len(stores_r)
96 A = len(areas_e)
97
98 # Build store -> area map (mode area in retail rows; fallback to first area)
99 mode_area = (
100     dfr.assign(area=dfr["area"].replace("", np.nan))
101     .groupby("store")["area"]
102     .agg(lambda x: x.mode().iat[0] if len(x.mode())>0 else np.nan)
103 )
104 fallback_area = areas_e[0]
105 store_to_area = {s: (mode_area.get(s) if pd.notna(mode_area.get(s)) else
106     ↪ fallback_area) for s in stores_r}
107 area_index = {a:i for i,a in enumerate(areas_e)}
108 M_sa = np.zeros((S, A), dtype=np.float32)
109 for si, s in enumerate(stores_r):
110     a = store_to_area.get(s, fallback_area)
111     if a in area_index:
112         M_sa[si, area_index[a]] = 1.0
113 M_sa_t = torch.tensor(M_sa, device=device)
114
115 # For covariates: per-entity metadata
116 prod_to_cat = dfx.groupby("product")["category"].agg(lambda x:
117     ↪ x.mode().iat[0]).to_dict()
118 area_to_region = dfx.groupby("area")["region"].agg(lambda x:
119     ↪ x.mode().iat[0]).to_dict()

```

```

114 store_to_region = dfr.groupby("store")["region"].agg(lambda x:
    ↪ x.mode().iat[0]).reindex(stores_r).to_dict()
115 store_to_size = dfr.groupby("store")["store_size"].agg(lambda x:
    ↪ x.mode().iat[0]).reindex(stores_r).to_dict()
116
117 # Unified covariate vocabulary
118 def vocab_from(field, values):
119     vals = sorted({str(v) for v in values if pd.notna(v) and str(v).strip()!=""})
120     return [(field, v) for v in vals]
121 vocab = []
122 vocab += vocab_from("channel", ["retail", "ecommerce"])
123 vocab += vocab_from("region", dfx["region"].unique())
124 vocab += vocab_from("area", areas_e)
125 vocab += vocab_from("store", stores_r)
126 vocab += vocab_from("store_size", dfx["store_size"].unique())
127 vocab += vocab_from("category", dfx["category"].unique())
128 vocab += vocab_from("product", prod_order)
129 tok2idx = {f"{k}::{v}": i for i, (k, v) in enumerate(vocab)}
130 D_cov = len(tok2idx)
131
132 def vec_for_tokens(tokens):
133     v = np.zeros(D_cov, dtype=np.float32)
134     for (k, val) in tokens:
135         key = f"{k}::{val}"
136         if key in tok2idx:
137             v[tok2idx[key]] = 1.0
138     return v
139
140 # Products: product + category
141 X_prod = np.stack([
142     vec_for_tokens([("product", p), ("category", prod_to_cat.get(p, ""))])
143     for p in prod_order
144 ], axis=0) # [P, D_cov]
145
146 # Stores (retail): store + area(store) + region(store) + size + channel=Retail
147 X_store = np.stack([
148     vec_for_tokens([
149         ("store", s),
150         ("area", store_to_area.get(s, fallback_area)),
151         ("region", store_to_region.get(s, "")),
152         ("store_size", store_to_size.get(s, "")),
153         ("channel", "retail"),
154     ])
155     for s in stores_r
156 ], axis=0) # [S, D_cov]

```

```

157
158 # Areas (ecomm): area + region(area) + Online + channel=Ecommerce
159 X_area = np.stack([
160     vec_for_tokens([
161         ("area", a),
162         ("region", area_to_region.get(a, "")),
163         ("store_size", "Online"),
164         ("channel", "ecommerce"),
165     ])
166     for a in areas_e
167 ], axis=0) # [A, D_cov]
168
169 X_prod_t = torch.tensor(X_prod, dtype=torch.float32, device=device)
170 X_store_t = torch.tensor(X_store, dtype=torch.float32, device=device)
171 X_area_t = torch.tensor(X_area, dtype=torch.float32, device=device)
172
173 # ----- build daily index + panels -----
174 full_idx = pd.date_range(dfx["date"].min(), dfx["date"].max(), freq="D")
175
176 # children (encoder inputs)
177 panel_prod_child = (
178     dfx.groupby(["date", "product"])["sales"].sum().unstack("product")
179     .reindex(full_idx).fillna(0.0).astype(np.float32)
180 )
181 for p in prod_order:
182     if p not in panel_prod_child.columns: panel_prod_child[p] = 0.0
183 panel_prod_child = panel_prod_child[prod_order]
184
185 panel_store_child = (
186     dfr.groupby(["date", "store"])["sales"].sum().unstack("store")
187     .reindex(full_idx).fillna(0.0).astype(np.float32)
188     .reindex(columns=stores_r, fill_value=0.0)
189 )
190
191 panel_area_child_e = (
192     dfe.groupby(["date", "area"])["sales"].sum().unstack("area")
193     .reindex(full_idx).fillna(0.0).astype(np.float32)
194     .reindex(columns=areas_e, fill_value=0.0)
195 )
196
197 # parents (observed)
198 panel_prod_tot = (
199     dfx.groupby(["date", "product"])["sales"].sum().unstack("product")
200     .reindex(full_idx).fillna(0.0).astype(np.float32)
201     .reindex(columns=prod_order, fill_value=0.0)

```

```

202     )
203     panel_cat_tot = pd.DataFrame({
204         "CAT_A": panel_prod_tot[["P1", "P2"]].sum(axis=1),
205         "CAT_B": panel_prod_tot[["P3", "P4"]].sum(axis=1),
206         "CAT_C": panel_prod_tot[["P5", "P6"]].sum(axis=1),
207     }, index=full_idx).astype(np.float32)[["CAT_A", "CAT_B", "CAT_C"]]
208     panel_chan_tot = (
209         dfx.groupby(["date", "channel_c"])["sales"].sum().unstack("channel_c")
210         .reindex(full_idx).fillna(0.0).astype(np.float32)
211     )
212     for ch in ["retail", "ecommerce"]:
213         if ch not in panel_chan_tot.columns: panel_chan_tot[ch] = 0.0
214     panel_chan_tot = panel_chan_tot[["retail", "ecommerce"]]
215
216     panel_store_tot = (
217         dfx.groupby(["date", "store"])["sales"].sum().unstack("store")
218         .reindex(full_idx).fillna(0.0).astype(np.float32)
219         .reindex(columns=stores_r, fill_value=0.0)
220     )
221     panel_area_tot_all = (
222         dfx.assign(area=dfx["area"].replace("", np.nan))
223         .groupby(["date", "area"])["sales"].sum().unstack("area")
224         .reindex(full_idx).fillna(0.0).astype(np.float32)
225     )
226     for a in areas_e:
227         if a not in panel_area_tot_all.columns: panel_area_tot_all[a] = 0.0
228     panel_area_tot = panel_area_tot_all[areas_e]
229     panel_gt = pd.DataFrame({"Grand_Total": dfx.groupby("date")["sales"].sum()
230                             .reindex(full_idx).fillna(0.0).astype(np.float32)})
231
232     # Align train/test split with df.scenario
233     test_start = test_dates.min()
234     if test_start not in full_idx:
235         raise ValueError("Test start date not on continuous daily index.")
236     train_end_pos = full_idx.get_loc(test_start) - 1
237     if train_end_pos < 0:
238         raise ValueError("Train window is empty (test begins at earliest date).")
239     train_days = train_end_pos + 1
240     H = min(len(test_dates), horizon_cap)
241
242     def split_arr(frame):
243         arr = frame.values.astype(np.float32)
244         return arr[:train_days, :], arr[train_days:train_days+H, :]
245
246     # children

```

```

247 Yp_tr, Yp_te = split_arr(panel_prod_child)
248 Ys_tr, Ys_te = split_arr(panel_store_child)
249 Ya_tr, Ya_te = split_arr(panel_area_child_e)
250 # parents
251 Y_prod_tr, Y_prod_te = split_arr(panel_prod_tot)
252 Y_cat_tr, Y_cat_te = split_arr(panel_cat_tot)
253 Y_chan_tr, Y_chan_te = split_arr(panel_chan_tot)
254 Y_store_tr, Y_store_te = split_arr(panel_store_tot)
255 Y_area_tr, Y_area_te = split_arr(panel_area_tot)
256 Y_gt_tr, Y_gt_te = split_arr(panel_gt)
257
258 # Normalization (train only)
259 def mu_sd(arr_tr):
260     mu = arr_tr.mean(axis=0).astype(np.float32)
261     sd = (arr_tr.std(axis=0)+1e-8).astype(np.float32)
262     return mu, sd
263 mu_p, sd_p = mu_sd(Yp_tr)
264 mu_s, sd_s = mu_sd(Ys_tr)
265 mu_a, sd_a = mu_sd(Ya_tr)
266 mu_prod, sd_prod = mu_sd(Y_prod_tr)
267 mu_cat, sd_cat = mu_sd(Y_cat_tr)
268 mu_chan, sd_chan = mu_sd(Y_chan_tr)
269 mu_store, sd_store = mu_sd(Y_store_tr)
270 mu_area, sd_area = mu_sd(Y_area_tr)
271 mu_gt, sd_gt = mu_sd(Y_gt_tr)
272
273 Yp_norm = (np.vstack([Yp_tr, Yp_te]) - mu_p[None,:]) / sd_p[None,:]
274 Ys_norm = (np.vstack([Ys_tr, Ys_te]) - mu_s[None,:]) / sd_s[None,:]
275 Ya_norm = (np.vstack([Ya_tr, Ya_te]) - mu_a[None,:]) / sd_a[None,:]
276
277 Y_prod_norm = (np.vstack([Y_prod_tr, Y_prod_te]) - mu_prod[None,:]) /
    ↪ sd_prod[None,:]
278 Y_cat_norm = (np.vstack([Y_cat_tr, Y_cat_te]) - mu_cat[None,:]) /
    ↪ sd_cat[None,:]
279 Y_chan_norm = (np.vstack([Y_chan_tr, Y_chan_te]) - mu_chan[None,:]) /
    ↪ sd_chan[None,:]
280 Y_store_norm = (np.vstack([Y_store_tr, Y_store_te]) - mu_store[None,:]) /
    ↪ sd_store[None,:]
281 Y_area_norm = (np.vstack([Y_area_tr, Y_area_te]) - mu_area[None,:]) /
    ↪ sd_area[None,:]
282 Y_gt_norm = (np.vstack([Y_gt_tr, Y_gt_te]) - mu_gt[None,:]) /
    ↪ sd_gt[None,:]
283
284 pairs = list(range(lookback, train_days - H + 1))
285

```

```

286 class AsymWindows(Dataset):
287     def __init__(self, Yp_n, Ys_n, Ya_n,
288                 Yprod_n, Ycat_n, Ychan_n, Ystore_n, Yarea_n, Ygt_n,
289                 pairs, L, H):
290         self.Yp, self.Ys, self.Ya = Yp_n, Ys_n, Ya_n
291         self.Yprod, self.Ycat, self.Ychan = Yprod_n, Ycat_n, Ychan_n
292         self.Ystore, self.Yarea, self.Ygt = Ystore_n, Yarea_n, Ygt_n
293         self.pairs, self.L, self.H = pairs, L, H
294     def __len__(self): return len(self.pairs)
295     def __getitem__(self, i):
296         t = self.pairs[i]
297         Xp = self.Yp[t-self.L:t, :].T.copy()      # [P,L]
298         Xs = self.Ys[t-self.L:t, :].T.copy()      # [S,L]
299         Xa = self.Ya[t-self.L:t, :].T.copy()      # [A,L]
300         Yprod = self.Yprod[t:t+self.H, :].T.copy() # [P,H]
301         Ycat = self.Ycat[t:t+self.H, :].T.copy()  # [3,H]
302         Ychan = self.Ychan[t:t+self.H, :].T.copy() # [2,H]
303         Ystore = self.Ystore[t:t+self.H, :].T.copy() # [S,H]
304         Yarea = self.Yarea[t:t+self.H, :].T.copy() # [A,H]
305         Ygt = self.Ygt[t:t+self.H, :].T.copy()    # [1,H]
306         return (torch.from_numpy(Xp), torch.from_numpy(Xs), torch.from_numpy(Xa),
307                 torch.from_numpy(Yprod), torch.from_numpy(Ycat),
308                 ↪ torch.from_numpy(Ychan),
309                 torch.from_numpy(Ystore), torch.from_numpy(Yarea),
310                 ↪ torch.from_numpy(Ygt))
311
312 train_ds = AsymWindows(Yp_norm, Ys_norm, Ya_norm,
313                        Y_prod_norm, Y_cat_norm, Y_chan_norm, Y_store_norm,
314                        ↪ Y_area_norm, Y_gt_norm,
315                        pairs, lookback, H)
316
317 train_dl = DataLoader(train_ds, batch_size=batch_size, shuffle=True, drop_last=True,
318                       num_workers=0, pin_memory=(device.type=="cuda"))
319
320 # ----- modules: encoder, FiLM, bias heads -----
321 class PositionalEncoding(nn.Module):
322     def __init__(self, d_model, max_len=4096):
323         super().__init__()
324         pe = torch.zeros(max_len, d_model)
325         pos = torch.arange(0, max_len).unsqueeze(1)
326         div = torch.exp(torch.arange(0, d_model, 2) * (-np.log(10000.0)/d_model))
327         pe[:,0::2] = torch.sin(pos*div); pe[:,1::2] = torch.cos(pos*div)
328         self.register_buffer("pe", pe.unsqueeze(0))
329     def forward(self, x): return x + self.pe[:, :x.size(1), :]
330
331 class PatchFront(nn.Module):

```

```

328     def __init__(self, L, Psz, Ssz, d_model, nhead, dff, nlayers, dropout=0.1):
329         super().__init__()
330         self.L, self.Psz, self.Ssz = L, Psz, Ssz
331         self.num_patches = 1 + (L - Psz)//Ssz
332         self.pos = PositionalEncoding(d_model)
333         enc = nn.TransformerEncoderLayer(
334             d_model=d_model, nhead=nhead, dim_feedforward=dff,
335             dropout=dropout, activation="gelu", batch_first=True, norm_first=True
336         )
337         self.encoder = nn.TransformerEncoder(enc, num_layers=nlayers)
338         self.out_norm = nn.LayerNorm(d_model)
339     def patchify(self, x): # x: [B*,L,1] -> [B*,Np,P]
340         return x.unfold(dimension=1, size=self.Psz, step=self.Ssz).squeeze(2)
341     def forward_with_patch(self, X, W_patch, b_patch, A=None):
342         B, C, L = X.shape
343         x = X.reshape(B*C, L).unsqueeze(-1)
344         patches = self.patchify(x).reshape(B, C, self.num_patches, self.Psz)
345         tok = torch.matmul(patches, W_patch.T) + b_patch.view(1,1,1,-1)
346         if A is not None:
347             tok = torch.einsum("ij,bcnd->bind", A, tok)
348         tok = tok.reshape(B*C, self.num_patches, -1)
349         tok = self.pos(tok)
350         z = self.encoder(tok).mean(dim=1)
351         z = self.out_norm(z).reshape(B, C, -1)
352         return z
353
354     class FiLM(nn.Module):
355         """one-hot (D_cov) -> (gamma, beta) in R^d_model; gamma = 1 +
356         ↪ scale*tanh(linear(...))."""
357         def __init__(self, D_cov, d_model, scale=film_scale):
358             super().__init__()
359             self.lin = nn.Linear(D_cov, 2*d_model)
360             nn.init.zeros_(self.lin.weight); nn.init.zeros_(self.lin.bias)
361             self.scale = scale
362             self.d = d_model
363         def forward(self, Xcov): # Xcov: [N, D_cov]
364             gb = self.lin(Xcov) # [N, 2d]
365             g, b = gb[:, :self.d], gb[:, self.d:]
366             gamma = 1.0 + self.scale * torch.tanh(g)
367             beta = b
368             return gamma, beta # [N,d], [N,d]
369
370     class BiasHead(nn.Module):
371         """one-hot (D_cov) -> horizon curve (H) via tiny bottleneck; mean-centered."""
372         def __init__(self, D_cov, H, bottleneck=bias_bottleneck, dropout=0.05):

```

```

372         super().__init__()
373         self.net = nn.Sequential(
374             nn.Linear(D_cov, bottleneck),
375             nn.GELU(),
376             nn.Dropout(dropout),
377             nn.Linear(bottleneck, H),
378         )
379         for m in self.net:
380             if isinstance(m, nn.Linear):
381                 nn.init.zeros_(m.weight); nn.init.zeros_(m.bias)
382         def forward(self, Xcov): # [N, D_cov] -> [N, H]
383             b = self.net(Xcov)
384             b = b - b.mean(dim=1, keepdim=True)
385             return b
386
387         encoder = PatchFront(lookback, patch_len, stride, d_model, nhead, dff, nlayers,
388                               ↪ dropout).to(device)
389         film_p = FiLM(D_cov, d_model).to(device)
390         film_s = FiLM(D_cov, d_model).to(device)
391         film_a = FiLM(D_cov, d_model).to(device)
392         bh_p = BiasHead(D_cov, H).to(device)
393         bh_s = BiasHead(D_cov, H).to(device)
394         bh_a = BiasHead(D_cov, H).to(device)
395
396         # ----- priors / constants -----
397         ALPHA_DECAY = 0.75
398         ALPHA_PATCH = 0.80
399         LAMBDA1_MIX, LAMBDA2_MIX = 0.30, 0.10
400         SIGMA0_PATCH, SIGMA0_BPATCH = 0.30, 0.10
401         SIGMA0_A_P, SIGMA0_A_S, SIGMA0_A_A = 0.30, 0.30, 0.30
402         SIGMA0_W_FACT, SIGMA0_B_MODE, SIGMA0_BO = 0.30, 0.20, 0.20
403         SIG_OBS_PROD, SIG_OBS_CAT, SIG_OBS_CHAN = 0.35, 0.30, 0.30
404         SIG_OBS_STORE, SIG_OBS_AREA, SIG_OBS_GT = 0.30, 0.30, 0.20
405
406         with torch.no_grad():
407             decay = 1.0 / torch.pow(torch.arange(1, H+1, device=device, dtype=torch.float32),
408                                     ↪ ALPHA_DECAY)
409             mu_prod_t, sd_prod_t = torch.tensor(mu_prod, device=device),
410                                     ↪ torch.tensor(sd_prod, device=device)
411             mu_cat_t, sd_cat_t = torch.tensor(mu_cat, device=device),
412                                     ↪ torch.tensor(sd_cat, device=device)
413             mu_chan_t, sd_chan_t = torch.tensor(mu_chan, device=device),
414                                     ↪ torch.tensor(sd_chan, device=device)
415             mu_store_t, sd_store_t = torch.tensor(mu_store, device=device),
416                                     ↪ torch.tensor(sd_store, device=device)

```

```

411     mu_area_t, sd_area_t = torch.tensor(mu_area, device=device),
    ↪ torch.tensor(sd_area, device=device)
412     mu_gt_t, sd_gt_t = torch.tensor(mu_gt, device=device),
    ↪ torch.tensor(sd_gt, device=device)
413     idx_A = torch.tensor([0,1], device=device)
414     idx_B = torch.tensor([2,3], device=device)
415     idx_C = torch.tensor([4,5], device=device)
416
417     # ----- pyro model & guide -----
418     def model(Xp, Xs, Xa, Yprod, Ycat, Ychan, Ystore, Yarea, Ygt):
419         pyro.module("encoder", encoder)
420         pyro.module("film_p", film_p); pyro.module("film_s", film_s);
    ↪ pyro.module("film_a", film_a)
421         pyro.module("bh_p", bh_p); pyro.module("bh_s", bh_s); pyro.module("bh_a", bh_a)
422
423         lam_p = torch.sigmoid(pyro.param("lambda_p", torch.tensor(0.0, device=device)))
424         lam_s = torch.sigmoid(pyro.param("lambda_s", torch.tensor(0.0, device=device)))
425         lam_a = torch.sigmoid(pyro.param("lambda_a", torch.tensor(0.0, device=device)))
426
427         B = Xp.shape[0]
428         Xp = Xp.to(device); Xs = Xs.to(device); Xa = Xa.to(device)
429         Yprod = Yprod.to(device); Ycat = Ycat.to(device); Ychan = Ychan.to(device)
430         Ystore = Ystore.to(device); Yarea = Yarea.to(device); Ygt = Ygt.to(device)
431
432         # Patch front (Bayesian columns)
433         sigma_patch = pyro.sample("sigma_patch",
    ↪ dist.LogNormal(torch.tensor(np.log(SIGMA0_PATCH), device=device),
    ↪ torch.tensor(0.3, device=device)))
434         sigma_bpatch = pyro.sample("sigma_bpatch",
    ↪ dist.LogNormal(torch.tensor(np.log(SIGMA0_BPATCH), device=device),
    ↪ torch.tensor(0.3, device=device)))
435         qpos = torch.arange(patch_len, device=device, dtype=torch.float32)
436         age = (patch_len - qpos)
437         col_scale = sigma_patch * (1.0 / (age ** ALPHA_PATCH))
438         W_patch = pyro.sample("W_patch", dist.Normal(torch.zeros((d_model, patch_len),
    ↪ device=device), col_scale.view(1,-1).expand(d_model, patch_len)).to_event(2))
439         b_patch = pyro.sample("b_patch", dist.Normal(torch.zeros((d_model,)),
    ↪ device=device), sigma_bpatch*torch.ones((d_model,)),
    ↪ device=device)).to_event(1))
440
441     def mixer(name, N, sig0):
442         sigma = pyro.sample(f"sigma_{name}",
    ↪ dist.LogNormal(torch.tensor(np.log(sig0), device=device),
    ↪ torch.tensor(0.3, device=device)))
443         g = torch.ones((N,N), device=device)

```

```

444         scale = sigma * (LAMBDA1_MIX*torch.eye(N, device=device) + LAMBDA2_MIX*g*(1 -
        ↪ torch.eye(N, device=device)))
445     return pyro.sample(name, dist.Normal(torch.zeros((N,N), device=device),
        ↪ scale).to_event(2))
446 A_p = mixer("A_p", P, SIGMA0_A_P)
447 A_s = mixer("A_s", S, SIGMA0_A_S)
448 A_a = mixer("A_a", A, SIGMA0_A_A)
449
450     # Encode modes
451 Zp = encoder.forward_with_patch(Xp, W_patch, b_patch, A=A_p)    # [B,P,d]
452 Zs = encoder.forward_with_patch(Xs, W_patch, b_patch, A=A_s)    # [B,S,d]
453 Za = encoder.forward_with_patch(Xa, W_patch, b_patch, A=A_a)    # [B,A,d]
454
455     # FiLM
456 gamma_p, beta_p = film_p(X_prod_t)
457 gamma_s, beta_s = film_s(X_store_t)
458 gamma_a, beta_a = film_a(X_area_t)
459 Zp = Zp * gamma_p[None, :, :] + beta_p[None, :, :]
460 Zs = Zs * gamma_s[None, :, :] + beta_s[None, :, :]
461 Za = Za * gamma_a[None, :, :] + beta_a[None, :, :]
462
463     # Factor maps
464 sigma_wf = pyro.sample("sigma_wf",
        ↪ dist.LogNormal(torch.tensor(np.log(SIGMA0_W_FACT), device=device),
        ↪ torch.tensor(0.4, device=device)))
465 Wp_f = pyro.sample("Wp_f", dist.Normal(torch.zeros((d_model, rank*H),
        ↪ device=device), sigma_wf*torch.ones((d_model, rank*H),
        ↪ device=device)).to_event(2))
466 Ws_f = pyro.sample("Ws_f", dist.Normal(torch.zeros((d_model, rank*H),
        ↪ device=device), sigma_wf*torch.ones((d_model, rank*H),
        ↪ device=device)).to_event(2))
467 Wa_f = pyro.sample("Wa_f", dist.Normal(torch.zeros((d_model, rank*H),
        ↪ device=device), sigma_wf*torch.ones((d_model, rank*H),
        ↪ device=device)).to_event(2))
468
469 Ap = torch.einsum("bpd,dr->bpr", Zp, Wp_f).view(B, P, rank, H) *
        ↪ decay.view(1,1,1,H)
470 Ss = torch.einsum("bsd,dr->bsr", Zs, Ws_f).view(B, S, rank, H) *
        ↪ decay.view(1,1,1,H)
471 Ae = torch.einsum("bad,dr->bar", Za, Wa_f).view(B, A, rank, H) *
        ↪ decay.view(1,1,1,H)
472
473     # Biases + covariate horizon biases (gated)

```

```

474 sigma_bm = pyro.sample("sigma_bm",
    ↳ dist.LogNormal(torch.tensor(np.log(SIGMA0_B_MODE), device=device),
    ↳ torch.tensor(0.3, device=device)))
475 bp = pyro.sample("bp", dist.Normal(torch.zeros((P,H), device=device),
    ↳ sigma_bm*torch.ones((P,H), device=device)).to_event(2))
476 bs = pyro.sample("bs", dist.Normal(torch.zeros((S,H), device=device),
    ↳ sigma_bm*torch.ones((S,H), device=device)).to_event(2))
477 ba = pyro.sample("ba", dist.Normal(torch.zeros((A,H), device=device),
    ↳ sigma_bm*torch.ones((A,H), device=device)).to_event(2))
478 b0 = pyro.sample("b0", dist.Normal(torch.zeros((H,), device=device),
    ↳ SIGMA0_B0*torch.ones((H,), device=device)).to_event(1))

479
480 bp_cov = bh_p(X_prod_t); bs_cov = bh_s(X_store_t); ba_cov = bh_a(X_area_t)
481 bp = bp + lam_p * bp_cov
482 bs = bs + lam_s * bs_cov
483 ba = ba + lam_a * ba_cov
484
485 # Compose leaves
486 Y_r = torch.einsum("bprh,bsrh->bps h", Ap, Ss) + bp[:,None,:] + bs[None,:,:] +
    ↳ b0[None,None,None,:]
487 Y_e = torch.einsum("bprh,barh->bpah", Ap, Ae) + bp[:,None,:] + ba[None,:,:] +
    ↳ b0[None,None,None,:]
488
489 # Parents by exact sums
490 Y_prod_den = Y_r.sum(dim=2) + Y_e.sum(dim=2) # [B,P,H]
491 Y_cat_den = torch.stack([Y_prod_den[:, idx_A, :].sum(dim=1),
492                          Y_prod_den[:, idx_B, :].sum(dim=1),
493                          Y_prod_den[:, idx_C, :].sum(dim=1)], dim=1) # [B,3,H]
494 Y_retail_tot = Y_r.sum(dim=(1,2)) # [B,H]
495 Y_ecomm_tot = Y_e.sum(dim=(1,2)) # [B,H]
496 Y_chan_den = torch.stack([Y_retail_tot, Y_ecomm_tot], dim=1) # [B,2,H]
497 Y_store_den = Y_r.sum(dim=1) # [B,S,H]
498 R_area = torch.einsum("bsh,sa->bah", Y_store_den, M_sa_t) # [B,A,H]
499 E_area = Y_e.sum(dim=1) # [B,A,H]
500 Y_area_den = R_area + E_area # [B,A,H]
501 Y_gt_den = Y_retail_tot + Y_ecomm_tot # [B,H]
502
503 # Normalize for likelihoods
504 Y_prod_hat = (Y_prod_den - mu_prod_t[None,:,None]) / sd_prod_t[None,:,None]
505 Y_cat_hat = (Y_cat_den - mu_cat_t[None,:,None]) / sd_cat_t[None,:,None]
506 Y_chan_hat = (Y_chan_den - mu_chan_t[None,:,None]) / sd_chan_t[None,:,None]
507 Y_store_hat = (Y_store_den - mu_store_t[None,:,None]) / sd_store_t[None,:,None]
508 Y_area_hat = (Y_area_den - mu_area_t[None,:,None]) / sd_area_t[None,:,None]
509 Y_gt_hat = (Y_gt_den - mu_gt_t.view(1)) / sd_gt_t.view(1)
510

```

```

511     # Likelihoods (parents only)
512     sig_prod = pyro.sample("sig_prod",
513         ↪ dist.LogNormal(torch.tensor(np.log(SIG_OBS_PROD), device=device),
514         ↪ torch.tensor(0.4, device=device)))
515     sig_cat = pyro.sample("sig_cat",
516         ↪ dist.LogNormal(torch.tensor(np.log(SIG_OBS_CAT), device=device),
517         ↪ torch.tensor(0.4, device=device)))
518     sig_chan = pyro.sample("sig_chan",
519         ↪ dist.LogNormal(torch.tensor(np.log(SIG_OBS_CHAN), device=device),
520         ↪ torch.tensor(0.4, device=device)))
521     sig_store = pyro.sample("sig_store",
522         ↪ dist.LogNormal(torch.tensor(np.log(SIG_OBS_STORE), device=device),
523         ↪ torch.tensor(0.4, device=device)))
524     sig_area = pyro.sample("sig_area",
525         ↪ dist.LogNormal(torch.tensor(np.log(SIG_OBS_AREA), device=device),
526         ↪ torch.tensor(0.4, device=device)))
527     sig_gt = pyro.sample("sig_gt",
528         ↪ dist.LogNormal(torch.tensor(np.log(SIG_OBS_GT), device=device),
529         ↪ torch.tensor(0.4, device=device)))
530
531     with pyro.plate("b_prod", B):
532         pyro.sample("Y_prod_totals", dist.Normal(Y_prod_hat, sig_prod).to_event(2),
533             ↪ obs=Yprod)
534     with pyro.plate("b_cat", B):
535         pyro.sample("Y_cat_totals", dist.Normal(Y_cat_hat, sig_cat).to_event(2),
536             ↪ obs=Ycat)
537     with pyro.plate("b_chan", B):
538         pyro.sample("Y_chan_totals", dist.Normal(Y_chan_hat, sig_chan).to_event(2),
539             ↪ obs=Ychan)
540     with pyro.plate("b_store", B):
541         pyro.sample("Y_store_totals", dist.Normal(Y_store_hat,
542             ↪ sig_store).to_event(2), obs=Ystore)
543     with pyro.plate("b_area", B):
544         pyro.sample("Y_area_totals", dist.Normal(Y_area_hat, sig_area).to_event(2),
545             ↪ obs=Yarea)
546     with pyro.plate("b_gt", B):
547         pyro.sample("Y_gt", dist.Normal(Y_gt_hat, sig_gt).to_event(1),
548             ↪ obs=Ygt.squeeze(1))
549
550 def guide(Xp, Xs, Xa, Yprod, Ycat, Ychan, Ystore, Yarea, Ygt):
551     def q_logn(name, med, su0=-1.0):
552         loc = pyro.param(f"{name}_loc", torch.tensor(np.log(med), device=device))
553         su = pyro.param(f"{name}_su", torch.tensor(su0, device=device))
554         sc = torch.nn.functional.softplus(su) + 1e-5
555         pyro.sample(name, dist.LogNormal(loc, sc))

```

```

538
539 for nm, med in [("sigma_patch", SIGMAO_PATCH), ("sigma_bpatch", SIGMAO_BPATCH),
540                 ("sigma_A_p", SIGMAO_A_P), ("sigma_A_s", SIGMAO_A_S), ("sigma_A_a", S
541                                     ↪ IGMAO_A_A),
542                 ("sigma_wf", SIGMAO_W_FACT), ("sigma_bm", SIGMAO_B_MODE),
543                 ("sig_prod", SIG_OBS_PROD), ("sig_cat", SIG_OBS_CAT), ("sig_chan", SI
544                                     ↪ G_OBS_CHAN),
545                 ("sig_store", SIG_OBS_STORE), ("sig_area", SIG_OBS_AREA), ("sig_gt",
546                                     ↪ SIG_OBS_GT)]:
547     q_logn(nm, med)
548
549 # Patch params
550 Wpatch_loc = pyro.param("W_patch_loc", torch.zeros((d_model, patch_len),
551                                     ↪ device=device))
552 Wpatch_su = pyro.param("W_patch_su", torch.full((d_model, patch_len), -1.5,
553                                     ↪ device=device))
554 pyro.sample("W_patch", dist.Normal(Wpatch_loc,
555                                     ↪ torch.nn.functional.softplus(Wpatch_su)+1e-5).to_event(2))
556
557 bpatch_loc = pyro.param("b_patch_loc", torch.zeros((d_model,), device=device))
558 bpatch_su = pyro.param("b_patch_su", torch.full((d_model,), -1.5,
559                                     ↪ device=device))
560 pyro.sample("b_patch", dist.Normal(bpatch_loc,
561                                     ↪ torch.nn.functional.softplus(bpatch_su)+1e-5).to_event(1))
562
563 # Mixers
564 def q_mixer(name, N):
565     A_loc = pyro.param(f"{name}_loc", torch.zeros((N,N), device=device))
566     A_su = pyro.param(f"{name}_su", torch.full((N,N), -1.5, device=device))
567     pyro.sample(name, dist.Normal(A_loc,
568                                     ↪ torch.nn.functional.softplus(A_su)+1e-5).to_event(2))
569 q_mixer("A_p", P); q_mixer("A_s", S); q_mixer("A_a", A)
570
571 # Factor maps
572 def q_fmap(name):
573     W_loc = pyro.param(f"{name}_loc", torch.zeros((d_model, rank*H),
574                                     ↪ device=device))
575     W_su = pyro.param(f"{name}_su", torch.full((d_model, rank*H), -1.5,
576                                     ↪ device=device))
577     pyro.sample(name, dist.Normal(W_loc,
578                                     ↪ torch.nn.functional.softplus(W_su)+1e-5).to_event(2))
579 q_fmap("Wp_f"); q_fmap("Ws_f"); q_fmap("Wa_f")
580
581 # Biases
582 def q_bias(name, shape):

```

```

571         loc = pyro.param(f"{name}_loc", torch.zeros(shape, device=device))
572         su = pyro.param(f"{name}_su", torch.full(shape, -1.5, device=device))
573         pyro.sample(name, dist.Normal(loc,
574             ↪ torch.nn.functional.softplus(su)+1e-5).to_event(len(shape)))
575
576         q_bias("bp", (P,H)); q_bias("bs", (S,H)); q_bias("ba", (A,H)); q_bias("b0", (H,))
577
578     # ----- helpers: forecast + quick val metrics -----
579     @torch.no_grad()
580     def _forecast():
581         encoder.eval(); film_p.eval(); film_s.eval(); film_a.eval()
582         bh_p.eval(); bh_s.eval(); bh_a.eval()
583
584         Xp_last = Yp_norm[train_days-lookback:train_days, :].T.copy()
585         Xs_last = Ys_norm[train_days-lookback:train_days, :].T.copy()
586         Xa_last = Ya_norm[train_days-lookback:train_days, :].T.copy()
587
588         Xp_t = torch.from_numpy(Xp_last).to(device).unsqueeze(0)
589         Xs_t = torch.from_numpy(Xs_last).to(device).unsqueeze(0)
590         Xa_t = torch.from_numpy(Xa_last).to(device).unsqueeze(0)
591
592     # posterior means
593     Wpatch_loc = pyro.param("W_patch_loc").to(device)
594     bpatch_loc = pyro.param("b_patch_loc").to(device)
595     Ap_loc = pyro.param("A_p_loc").to(device)
596     As_loc = pyro.param("A_s_loc").to(device)
597     Aa_loc = pyro.param("A_a_loc").to(device)
598     Wp_f_loc = pyro.param("Wp_f_loc").to(device)
599     Ws_f_loc = pyro.param("Ws_f_loc").to(device)
600     Wa_f_loc = pyro.param("Wa_f_loc").to(device)
601     bp_loc = pyro.param("bp_loc").to(device)
602     bs_loc = pyro.param("bs_loc").to(device)
603     ba_loc = pyro.param("ba_loc").to(device)
604     b0_loc = pyro.param("b0_loc").to(device)
605
606     Zp = encoder.forward_with_patch(Xp_t, Wpatch_loc, bpatch_loc, A=Ap_loc)
607     Zs = encoder.forward_with_patch(Xs_t, Wpatch_loc, bpatch_loc, A=As_loc)
608     Za = encoder.forward_with_patch(Xa_t, Wpatch_loc, bpatch_loc, A=Aa_loc)
609
610     gamma_p, beta_p = film_p(X_prod_t); Zp = Zp * gamma_p[None,:::] +
611         ↪ beta_p[None,:::]
612     gamma_s, beta_s = film_s(X_store_t); Zs = Zs * gamma_s[None,:::] +
613         ↪ beta_s[None,:::]
614     gamma_a, beta_a = film_a(X_area_t); Za = Za * gamma_a[None,:::] +
615         ↪ beta_a[None,:::]

```

```

612     Ap = torch.einsum("bpd,dr->bpr", Zp, Wp_f_loc).view(1, P, rank, H) *
        ↳ decay.view(1,1,1,H)
613     Ss = torch.einsum("bsd,dr->bsr", Zs, Ws_f_loc).view(1, S, rank, H) *
        ↳ decay.view(1,1,1,H)
614     Ae = torch.einsum("bad,dr->bar", Za, Wa_f_loc).view(1, A, rank, H) *
        ↳ decay.view(1,1,1,H)

615
616     Y_r = torch.einsum("bprh,bsrh->bps", Ap, Ss) + bp_loc[:,None,:] +
        ↳ bs_loc[None,:,:] + b0_loc[None,None,None,:]
617     Y_e = torch.einsum("bprh,barh->bpah", Ap, Ae) + bp_loc[:,None,:] +
        ↳ ba_loc[None,:,:] + b0_loc[None,None,None,:]

618
619     return Y_r.squeeze(0).cpu().numpy(), Y_e.squeeze(0).cpu().numpy() # [P,S,H],
        ↳ [P,A,H]

620
621 def _mape(a, b, eps=1e-8):
622     a = np.asarray(a, dtype=float); b = np.asarray(b, dtype=float)
623     return float(np.mean(np.abs(a-b) / np.maximum(eps, np.abs(a))) * 100.0)
624
625 def _val_metrics():
626     Y_r, Y_e = _forecast() # [P,S,H], [P,A,H]
627
628     # parents from predictions
629     pred_prod = Y_r.sum(axis=1) + Y_e.sum(axis=1) # [P,H]
630     pred_cat = np.stack([
631         pred_prod[[0,1], :].sum(axis=0),
632         pred_prod[[2,3], :].sum(axis=0),
633         pred_prod[[4,5], :].sum(axis=0),
634     ], axis=0) # [3,H]
635     pred_chan = np.stack([Y_r.sum(axis=(0,1)), Y_e.sum(axis=(0,1))], axis=0) # [2,H]
636     pred_gt = pred_chan.sum(axis=0, keepdims=True) # [1,H]
637
638     # ground truths (Test slice)
639     gt_prod = Y_prod_te[:H, :].T # [P,H]
640     gt_cat = Y_cat_te[:H, :].T # [3,H]
641     gt_chan = Y_chan_te[:H, :].T # [2,H]
642     gt_gt = Y_gt_te[:H, :].T # [1,H]
643
644     m_gt = _mape(gt_gt, pred_gt)
645     m_prod = float(np.mean([_mape(gt_prod[i], pred_prod[i]) for i in range(P)]))
646     m_cat = float(np.mean([_mape(gt_cat[i], pred_cat[i]) for i in range(3)]))
647     m_chan = float(np.mean([_mape(gt_chan[i], pred_chan[i]) for i in range(2)]))
648     return m_gt, m_prod, m_cat, m_chan
649
650 # ----- train (with logging) -----

```

```

651     pyro.clear_param_store()
652
653     optimizer = ExponentialLR(
654     {
655         "optimizer": torch.optim.AdamW,
656         "optim_args": {"lr": lr}, # starting LR
657         "gamma": 0.95 # multiply LR by 0.95 each epoch
658     },
659     clip_args={"clip_norm": 1.0}
660 )
661
662     svi = SVI(model, guide, optimizer, loss=Trace_ELBO())
663
664     train_log = []
665     encoder.train(); film_p.train(); film_s.train(); film_a.train()
666     bh_p.train(); bh_s.train(); bh_a.train()
667     for ep in range(1, epochs+1):
668         total = 0.0
669         n_batches = 0
670         for (Xp, Xs, Xa, Yprod, Ycat, Ychan, Ystore, Yarea, Ygt) in train_dl:
671             Xp, Xs, Xa = Xp.to(device), Xs.to(device), Xa.to(device)
672             Yprod, Ycat, Ychan = Yprod.to(device), Ycat.to(device), Ychan.to(device)
673             Ystore, Yarea, Ygt = Ystore.to(device), Yarea.to(device), Ygt.to(device)
674             loss = svi.step(Xp, Xs, Xa, Yprod, Ycat, Ychan, Ystore, Yarea, Ygt)
675             total += loss
676             n_batches += 1
677
678         elbo = -total / max(1, n_batches)
679         # quick validation on Test parents (denorm) , cheap and informative
680         m_gt, m_prod, m_cat, m_chan = _val_metrics()
681         train_log.append({
682             "epoch": ep,
683             "ELBO": float(elbo),
684             "MAPE_GT_%": m_gt,
685             "MAPE_Products_mean_%": m_prod,
686             "MAPE_Categories_mean_%": m_cat,
687             "MAPE_Channels_mean_%": m_chan,
688         })
689         print(f"Epoch {ep:03d} | ELBO: {elbo:10.4f} | "
690             f"Val MAPE GT: {m_gt:6.2f}% | Prod: {m_prod:6.2f}% | "
691             f"Cat: {m_cat:6.2f}% | Chan: {m_chan:6.2f}%")
692
693     # ----- forecast & append -----
694     Y_r, Y_e = _forecast()
695

```

```

696 meta_cols = ["series", "channel", "region", "area", "store", "store_size", "category", "pro
    ↪ duct", "channel_c"]
697 meta = dfx[meta_cols].drop_duplicates("series")
698
699 r_map = (meta[meta["channel_c"].eq("retail")]
700         .loc[:, ["series", "store", "product"]]
701         .drop_duplicates()
702         .set_index(["store", "product"])["series"])
703 e_map = (meta[meta["channel_c"].eq("ecommerce")]
704         .loc[:, ["series", "area", "product"]]
705         .drop_duplicates()
706         .set_index(["area", "product"])["series"])
707
708 test_series = set(dfx.loc[dfx["scenario"]=="Test", "series"].unique())
709 rows = []
710
711 # Retail leaves
712 for p_idx, p in enumerate(prod_order):
713     for s_idx, s in enumerate(stores_r):
714         key = (s, p)
715         if key not in r_map.index:
716             continue
717         ser = r_map.loc[key]
718         if ser not in test_series:
719             continue
720         vals = Y_r[p_idx, s_idx, :] # [H]
721         for d, y in zip(test_dates[:H], vals):
722             rows.append((d, ser, y))
723
724 # Ecommerce leaves
725 for p_idx, p in enumerate(prod_order):
726     for a_idx, a in enumerate(areas_e):
727         key = (a, p)
728         if key not in e_map.index:
729             continue
730         ser = e_map.loc[key]
731         if ser not in test_series:
732             continue
733         vals = Y_e[p_idx, a_idx, :] # [H]
734         for d, y in zip(test_dates[:H], vals):
735             rows.append((d, ser, y))
736
737 if len(rows) == 0:
738     raise ValueError("No mapped leaves matched Test series; check
    ↪ series/channel/store/area naming.")

```

```

739
740 fc_long = pd.DataFrame(rows, columns=["date", "series", "sales"])
741 fc_long["scenario"] = "hbptst_forecast"
742
743 # Merge hierarchy keys
744 fc_long = fc_long.merge(dfx[["series", "channel", "region", "area", "store", "store_size"]
↪  , "category", "product"]].drop_duplicates("series"),
745                               on="series", how="left")
746
747 # exist_flag for forecasts; zero if forecast date < first live
748 first_live = dfx[dfx["exist_flag"]==1].groupby("series")["date"].min()
749 fc_long["exist_flag"] = (fc_long["date"] >=
↪  fc_long["series"].map(first_live)).astype(int).fillna(0)
750 pre_intro = fc_long["exist_flag"].eq(0)
751 if pre_intro.any():
752     fc_long.loc[pre_intro, "sales"] = 0.0
753
754 # Final order & append
755 ordered = ["scenario", "date", "series", "exist_flag", "channel", "region", "area",
756           "store", "store_size", "category", "product", "sales"]
757 fc_long = fc_long[ordered]
758 out = pd.concat([df, fc_long], ignore_index=True)
759
760 # training log DF
761 log_df = pd.DataFrame(train_log, columns=["epoch", "ELBO", "MAPE_GT%", "MAPE_Products_"]
↪  mean%", "MAPE_Categories_mean%", "MAPE_Channels_mean%"])
762 return out, log_df

```
